

DATA VISUALIZATION WITH PYTHON

COURSE NOTES

Transforming Data into
Meaningful Visual Stories



Visualize

Create insightful charts and plots using Python libraries



Explore

Build interactive dashboards and advanced visualizations



Analyze

Perform spatial data analysis and create maps



Communicate

Tell compelling data stories and drive better decisions



TOOLS & LIBRARIES

matplotlib



seaborn



plotly



Dash



folium

PREPARED BY

KASHIF MAQBOOL JOIYA

Data Scientist | AI Engineer



GitHub: <https://github.com/KashifMaqbool>

“ Good visualization is not just about making things look beautiful,
it's about making data meaningful. ”

Table of Contents

1	OVERVIEW OF DATA VISUALIZATION	1
1.1	INTRODUCTION TO DATA VISUALIZATION	1
1.2	TYPES OF DATA VISUALIZATION	1
1.2.1	BASIC CHARTS AND GRAPHS	1
1.2.2	INTERACTIVE DASHBOARDS AND MAPS	1
1.3	IMPORTANCE OF DATA VISUALIZATION	2
1.3.1	SIMPLIFIES COMPLEX DATA	2
1.3.2	ENHANCES COMMUNICATION	2
1.3.3	HELPS IDENTIFY TRENDS AND PATTERNS	2
1.4	REAL-WORLD EXAMPLES OF DATA VISUALIZATION	2
1.4.1	COVID-19 VISUALIZATION BY THE NEW YORK TIMES	2
1.4.2	AIRBNB SMART PRICING FRAMEWORK	3
1.4.3	SPOTIFY PIE	3
1.4.4	NETFLIX INSIGHT TOOL DASHBOARD	3
1.5	APPLICATIONS OF DATA VISUALIZATION	3
1.5.1	BUSINESS	3
1.5.2	HEALTHCARE	4
1.5.3	EDUCATION	4
1.5.4	GOVERNMENT	4
1.5.5	SCIENCE AND RESEARCH	4
1.5.6	ENTERTAINMENT INDUSTRY	4
1.6	BEST PRACTICES FOR DATA VISUALIZATION	4
1.6.1	CHOOSING THE RIGHT VISUALIZATION TYPE	5
1.6.2	KEEP VISUALIZATIONS SIMPLE	5
1.6.3	USE CLEAR LABELS AND CONTEXT	5
1.6.4	MINIMIZE CLUTTER	5
1.6.5	CONSIDER THE AUDIENCE	5
1.7	GOOD VS BAD DATA VISUALIZATION	6
1.7.1	CHARACTERISTICS OF BAD VISUALIZATION	6
1.7.2	CHARACTERISTICS OF GOOD VISUALIZATION	6
1.8	DARKHORSE ANALYTICS	6
1.9	KEY TAKEAWAYS	6
2	TYPES OF PLOTS	7
2.1	INTRODUCTION TO DATA VISUALIZATION	7
2.2	LINE PLOT	8
2.2.1	USES OF LINE PLOTS	8
2.2.2	IMPORTANT CONSIDERATIONS	8
2.3	BAR PLOT	9
2.3.1	USES OF BAR PLOTS	9
2.3.2	ADVANTAGES OF BAR PLOTS	9

2.3.3	IMPORTANT CONSIDERATIONS	9
2.4	SCATTER PLOT	10
2.4.1	USES OF SCATTER PLOTS	10
2.4.2	OUTLIER HANDLING	10
2.5	BOX PLOT	11
2.5.1	COMPONENTS OF A BOX PLOT	11
2.5.2	USES OF BOX PLOTS	11
2.5.3	IMPORTANT CONSIDERATIONS	11
2.6	HISTOGRAM	12
2.6.1	USES OF HISTOGRAMS	12
2.6.2	CHOOSING THE RIGHT NUMBER OF BINS	13
2.7	KEY TAKEAWAYS	13
3	<u>PLOT LIBRARIES IN PYTHON</u>	14
3.1	INTRODUCTION TO PLOT LIBRARIES	14
3.2	MATPLOTLIB	14
3.2.1	FEATURES OF MATPLOTLIB	14
3.2.2	ADVANTAGES OF MATPLOTLIB	15
3.3	PANDAS PLOTTING LIBRARY	15
3.3.1	FEATURES OF PANDAS PLOTTING	15
3.3.2	ADVANTAGES OF PANDAS VISUALIZATION	15
3.4	SEABORN	15
3.4.1	FEATURES OF SEABORN	16
3.4.2	INTEGRATION WITH PANDAS	16
3.4.3	ADVANTAGES OF SEABORN	16
3.5	FOLIUM	16
3.5.1	TYPES OF MAPS IN FOLIUM	16
3.5.2	FEATURES OF FOLIUM	17
3.6	PLOTLY	17
3.6.1	TYPES OF VISUALIZATIONS IN PLOTLY	17
3.6.2	FEATURES OF PLOTLY	17
3.7	PYWAFFLE	17
3.7.1	TYPES OF VISUALIZATIONS IN PYWAFFLE	17
3.7.2	ADVANTAGES OF PYWAFFLE	18
3.8	SUMMARY OF PLOT LIBRARIES	18
3.8.1	MATPLOTLIB	18
3.8.2	PANDAS	18
3.8.3	SEABORN	18
3.8.4	FOLIUM	18
3.8.5	PLOTLY	18
3.8.6	PYWAFFLE	18
4	<u>INTRODUCTION TO MATPLOTLIB</u>	19
4.1	WHAT IS MATPLOTLIB?	19

4.2	HISTORY AND ORIGIN	19
4.3	USES OF MATPLOTLIB	19
4.4	MATPLOTLIB ARCHITECTURE	19
4.4.1	BACKEND LAYER	20
4.4.2	ARTIST LAYER	20
4.4.3	SCRIPTING LAYER (PYPLOT)	21
4.5	GENERATING A HISTOGRAM USING PYPLOT	21
4.5.1	EXAMPLE CODE	22
4.6	EXPLANATION OF THE HISTOGRAM CODE	22
4.6.1	IMPORTING LIBRARIES	22
4.6.2	GENERATING RANDOM NUMBERS	22
4.6.3	CREATING THE HISTOGRAM	22
4.6.4	DISPLAYING THE PLOT	22
4.7	ANATOMY OF A PLOT	22
4.7.1	MAIN COMPONENTS OF A PLOT	23
4.7.2	ADDITIONAL PLOT COMPONENTS	24
4.7.3	PLOT CUSTOMIZATION	24
4.8	SUMMARY	24
5	<u>BASIC PLOTTING WITH MATPLOTLIB</u>	<u>25</u>
5.1	INTRODUCTION TO MATPLOTLIB	25
5.2	JUPYTER NOTEBOOK AND MATPLOTLIB	25
5.3	USING THE SCRIPTING INTERFACE	25
5.4	CREATING A BASIC PLOT	26
5.5	USING THE %MATPLOTLIB INLINE MAGIC FUNCTION	26
5.5.1	IMPORTANT NOTE	26
5.6	NOTEBOOK BACKEND	27
5.6.1	HOW IT WORKS	27
5.6.2	ADVANTAGE OF NOTEBOOK BACKEND	27
5.7	MATPLOTLIB INTEGRATION WITH PANDAS	27
5.8	CREATING A LINE PLOT USING PANDAS	27
5.9	CREATING A HISTOGRAM USING PANDAS	28
5.10	KEY POINTS	28
6	<u>DATASET ON IMMIGRATION TO CANADA</u>	<u>29</u>
6.1	OVERVIEW	29
6.2	STRUCTURE OF THE DATASET	29
6.2.1	IMPORTANT PARTS OF THE DATASET	29
6.3	USING PANDAS FOR DATA ANALYSIS	29
6.4	REQUIRED LIBRARIES	30
6.4.1	IMPORTING LIBRARIES	30
6.5	READING THE EXCEL FILE	30
6.5.1	READING DATA INTO A DATAFRAME	30
6.5.2	DISPLAYING THE FIRST FIVE ROWS	30

6.6	PROCESSING THE DATAFRAME	30
6.6.1	SETTING COUNTRY NAME AS THE INDEX	30
6.7	ADDING TOTAL IMMIGRATION COLUMN	30
6.7.1	CREATING TOTAL COLUMN	30
6.8	NAMING THE DATAFRAME	31
6.9	SUMMARY	31
7	<u>LINE PLOTS</u>	31
7.1	INTRODUCTION TO LINE PLOTS	31
7.2	WHEN TO USE A LINE PLOT	31
7.3	EXAMPLE: IMMIGRATION FROM HAITI TO CANADA	32
7.4	UNDERSTANDING THE DATASET	32
7.4.1	DATASET STRUCTURE	32
7.5	DATAFRAME PROCESSING	32
7.6	GENERATING A LINE PLOT	33
7.6.1	STEP 1: IMPORT REQUIRED LIBRARIES	33
7.6.2	STEP 2: GENERATE THE LINE PLOT	33
7.6.3	STEP 3: ADD TITLE AND AXIS LABELS	33
7.6.4	STEP 4: DISPLAY THE PLOT	33
7.7	IMPORTANT NOTE	33
7.8	SUMMARY	33
8	<u>SUMMARY: INTRODUCTION TO DATA VISUALIZATION TOOLS</u>	34
8.1	OVERVIEW OF DATA VISUALIZATION	34
8.2	COMMON TYPES OF DATA VISUALIZATIONS	34
8.2.1	LINE PLOTS	34
8.2.2	BAR PLOTS	34
8.2.3	SCATTER PLOTS	34
8.2.4	BOX PLOTS	35
8.2.5	HISTOGRAMS	35
8.3	DATA VISUALIZATION LIBRARIES	35
8.3.1	MATPLOTLIB	35
8.3.2	MATPLOTLIB ARCHITECTURE	35
8.3.3	ANATOMY OF A PLOT	35
8.3.4	PANDAS	36
8.3.5	SEABORN	36
8.3.6	FOLIUM	36
8.3.7	PLOTLY	36
8.3.8	PYWAFFLE	36
8.4	JUPYTER NOTEBOOK	37
8.5	MATPLOTLIB BACKENDS	37
9	<u>AREA PLOTS</u>	37

9.1	UNDERSTANDING THE DATASET	37
9.2	FINDING THE TOP IMMIGRATION COUNTRIES	38
9.3	PREPARING DATA FOR AREA PLOTS	38
9.3.1	CREATING A DATAFRAME FOR THE TOP FIVE COUNTRIES	38
9.3.2	WHY TRANSPOSE THE DATAFRAME?	38
9.4	GENERATING AN AREA PLOT USING MATPLOTLIB	39
9.5	EXAMPLE CODE	39
9.6	UNDERSTANDING THE AREA PLOT	40
9.7	APPLICATIONS OF AREA PLOTS	40
9.8	SUMMARY	40
10	HISTOGRAMS	41
10.1	WHAT IS A HISTOGRAM?	41
10.2	UNDERSTANDING HISTOGRAM BINS	41
10.3	PREPARING THE DATA FOR VISUALIZATION	41
10.4	VISUALIZING IMMIGRATION DISTRIBUTION FOR 2013	42
10.5	CREATING A HISTOGRAM USING MATPLOTLIB	42
10.6	ISSUE WITH DEFAULT HISTOGRAM BINS	42
10.7	IMPROVING HISTOGRAM ALIGNMENT USING NUMPY	42
10.8	SUMMARY	44
11	BAR CHARTS	44
11.1	INTRODUCTION TO BAR CHARTS	44
11.1.1	EXAMPLE	44
11.2	CREATING A BAR CHART USING MATPLOTLIB	44
11.2.1	STRUCTURE OF DF_CANADA	44
11.3	GENERATING A BAR CHART	45
11.3.1	EXAMPLE CODE	45
11.4	OBSERVATIONS FROM THE BAR CHART	45
11.5	HORIZONTAL BAR CHARTS	46
11.5.1	EXAMPLE	46
11.6	CUSTOMIZING BAR COLORS	46
11.6.1	HIGHLIGHTING SPECIFIC YEARS	46
11.6.2	EXAMPLE	46
11.7	CHANGING BAR BORDER COLORS	46
11.7.1	EXAMPLE	46
11.8	SUMMARY	46
12	PIE CHARTS	47
12.1	WHAT IS A PIE CHART?	47
12.1.1	EXAMPLE	47
12.2	CREATING A PIE CHART USING MATPLOTLIB	47
12.2.1	DATASET EXAMPLE	47

12.2.2	STEP 1: GROUP THE DATA BY CONTINENT	47
12.2.3	RESULTING DATAFRAME	47
12.2.4	STEP 2: IMPORT MATPLOTLIB	48
12.2.5	STEP 3: CREATE THE PIE CHART	48
12.2.6	OUTPUT	48
12.3	EXPLODING PIE CHART SLICES	48
12.3.1	PURPOSE OF EXPLODE	48
12.3.2	HOW IT WORKS	48
12.3.3	EXAMPLE	48
12.4	LIMITATIONS OF PIE CHARTS	49
12.4.1	WHY BAR CHARTS ARE OFTEN PREFERRED	49
12.5	SUMMARY	49
13	BOX PLOTS	49
13.1	WHAT IS A BOX PLOT?	49
13.1.1	FIVE MAIN DIMENSIONS OF A BOX PLOT	49
13.1.2	OUTLIERS	50
13.2	CREATING A BOX PLOT USING MATPLOTLIB	50
13.2.1	PREPARE THE DATA	50
13.2.2	IMPORT REQUIRED LIBRARIES	51
13.2.3	CREATE A DATAFRAME FOR JAPAN	51
13.2.4	GENERATE THE BOX PLOT	51
13.2.5	COMPLETE THE FIGURE	51
13.3	INTERPRETATION OF THE BOX PLOT	51
13.3.1	OBSERVATIONS	51
13.4	KEY TAKEAWAYS	51
14	SCATTER PLOTS	52
14.1	WHAT IS A SCATTER PLOT?	52
14.1.1	EXAMPLE: INCOME VS. EDUCATION	52
14.2	CREATING A SCATTER PLOT WITH MATPLOTLIB	52
14.2.1	PREPARING THE DATA	52
14.2.2	IMPORTING MATPLOTLIB	52
14.2.3	CREATING THE SCATTER PLOT	52
14.3	INTERPRETING THE SCATTER PLOT	53
14.4	CUSTOMIZING SCATTER PLOTS	53
14.4.1	USING THE COLOR PARAMETER	53
14.4.2	USING THE SIZE PARAMETER (S)	54
14.5	KEY POINTS	54
15	UNDERSTANDING TREEMAPS AND PIVOT CHARTS	55
15.1	TREEMAPS	55
15.1.1	INTRODUCTION	55

15.1.2	APPLICATIONS OF TREEMAPS	55
15.1.3	IMPORTANCE OF TREEMAPS IN DATA VISUALIZATION	55
15.1.4	SYNTAX FOR GENERATING A TREEMAP	56
15.1.5	PRACTICAL EXAMPLE: VISUALIZING SALES DATA USING A TREEMAP	56
15.1.6	CONCLUSION	57
15.2	PIVOT CHARTS	57
15.2.1	INTRODUCTION	57
15.2.2	APPLICATIONS OF PIVOT CHARTS	58
15.2.3	IMPORTANCE OF PIVOT CHARTS IN DATA VISUALIZATION	58
15.2.4	SAMPLE DATA STRUCTURE	58
15.2.5	SYNTAX FOR GENERATING PIVOT CHARTS IN PYTHON	59
15.3	PRACTICAL EXAMPLE: VISUALIZING SALES DATA USING A PIVOT CHART	59
15.3.1	SAMPLE DATASET	59
15.3.2	CODE USED TO GENERATE THE DATASET	60
15.3.3	CREATE A PIVOT TABLE	61
15.3.4	GENERATE PIVOT CHART	61
15.3.5	OUTPUT	61
15.3.6	CONCLUSION	61
16	PLOTTING DIRECTLY WITH MATPLOTLIB	62
16.1	INTRODUCTION TO MATPLOTLIB	62
16.2	IMPORTING REQUIRED LIBRARIES	62
16.2.1	PURPOSE OF EACH LIBRARY	62
16.3	CREATING A FIGURE AND AXES	62
16.4	CREATING A LINE PLOT	62
16.4.1	GENERATING SAMPLE DATA	62
16.4.2	EXPLANATION	62
16.4.3	PLOTTING THE DATA	62
16.5	CREATING A SCATTER PLOT	63
16.6	ADDING TITLES AND AXIS LABELS	63
16.6.1	USING PYPLOT FUNCTIONS	63
16.6.2	USING AXES METHODS	63
16.7	SETTING AXIS LIMITS	63
16.8	ADDING GRID LINES	63
16.9	ADDING A LEGEND	63
16.10	CUSTOMIZING PLOTS	64
16.10.1	MARKER STYLES	64
16.10.2	COLOR CUSTOMIZATION	64
16.10.3	LINE STYLES	64
16.11	CREATING A BAR PLOT	64
16.11.1	EXPLANATION	65
16.12	CREATING A HISTOGRAM	65
16.12.1	PARAMETERS	65
16.13	CREATING A PIE CHART	65
16.13.1	PARAMETERS	65
16.14	CREATING MULTIPLE PLOTS ON THE SAME FIGURE	65

16.14.1	USING SUBPLOTS()	65
16.14.2	EXPLANATION	66
16.14.3	PLOTTING ON INDIVIDUAL AXES	66
16.15	CREATING MULTIPLE SUBPLOTS USING ADD_SUBPLOT()	66
16.15.1	SYNTAX	66
16.15.2	EXAMPLE	66
16.15.3	EXPLANATION	66
16.16	DATA STORYTELLING VS DATA VISUALIZATION	66
16.16.1	DATA STORYTELLING	66
16.16.2	DATA VISUALIZATION	67
16.16.3	RELATIONSHIP BETWEEN THE TWO	67
16.17	KEY TAKEAWAYS	67
17	SUMMARY: BASIC AND SPECIALIZED VISUALIZATION TOOLS	68
17.1	KEY CONCEPTS COVERED	68
17.1.1	PIE CHARTS	68
17.1.2	BOX PLOTS	68
17.1.3	SCATTER PLOTS	68
17.1.4	MATPLOTLIB	68
17.1.5	DATA STORYTELLING	68
18	WAFFLE CHARTS AND WORD CLOUD	69
18.1	WAFFLE CHARTS	69
18.1.1	INTRODUCTION	69
18.1.2	USE CASES OF WAFFLE CHARTS	70
18.1.3	APPLICATIONS OF WAFFLE CHARTS	70
18.1.4	CREATING WAFFLE CHARTS IN PYTHON	71
18.2	WORD CLOUD	71
18.2.1	INTRODUCTION	71
18.2.2	HOW WORD CLOUD WORKS	71
18.2.3	USE CASES OF WORD CLOUD	72
18.2.4	APPLICATIONS OF WORD CLOUD	72
18.3	KEY TAKEAWAYS	72
19	SEABORN AND REGRESSION PLOTS	73
19.1	INTRODUCTION TO SEABORN	73
19.2	REGRESSION PLOTS USING SEABORN	73
19.2.1	IMPORT SEABORN	74
19.2.2	CREATE A REGRESSION PLOT	74
19.3	CUSTOMIZING A REGRESSION PLOT	74
19.3.1	CHANGE THE COLOR	74
19.3.2	CHANGE THE MARKER SHAPE	74
19.4	PLOTTING CATEGORICAL DATA	74

19.4.1	COUNT PLOT	74
19.5	BAR PLOT FOR CATEGORICAL DATA	75
19.6	KEY TAKEAWAYS	75
20	INTRODUCTION TO FOLIUM	76
20.1	WHAT IS FOLIUM?	76
20.2	CREATING A BASIC MAP WITH FOLIUM	76
20.3	INTERACTIVE NATURE OF FOLIUM MAPS	76
20.4	DEFAULT MAP STYLE	77
20.5	CREATING A WORLD MAP CENTERED AROUND CANADA	77
20.5.1	THE ZOOM_START PARAMETER	77
20.6	DIFFERENT MAP STYLES IN FOLIUM	77
20.6.1	STAMEN TONER MAP	77
20.6.2	STAMEN TERRAIN MAP	78
20.7	KEY TAKEAWAYS	78
21	MAPS WITH MARKERS	78
21.1	CREATING A BASIC MAP WITH FOLIUM	78
21.1.1	STEPS	78
21.1.2	IMPORTANT PARAMETERS	79
21.2	UNDERSTANDING MARKERS	79
21.3	ADDING A SINGLE MARKER	79
21.3.1	CREATING A MARKER	79
21.3.2	EXPLANATION	79
21.4	CREATING MARKERS USING FEATURE GROUPS	79
21.4.1	STEP 1: CREATE A FEATURE GROUP	79
21.4.2	STEP 2: CREATE A CHILD ELEMENT	80
21.4.3	STEP 3: ADD THE FEATURE GROUP TO THE MAP	80
21.4.4	ADDING A LABEL TO THE MARKER	80
21.5	DISPLAYING MULTIPLE MARKERS	80
21.5.1	STEPS	81
21.6	USING MARKER CLUSTERS	81
21.6.1	CREATING A MARKER CLUSTER	81
21.6.2	HOW MARKER CLUSTERS WORK	81
21.6.3	ADVANTAGES OF MARKER CLUSTERING	81
21.7	REAL-WORLD APPLICATION	81
21.8	KEY TAKEAWAYS	82
22	CHOROPLETH MAPS	82
22.1	INTRODUCTION TO CHOROPLETH MAPS	82
22.2	EXAMPLES OF CHOROPLETH MAPS	82
22.2.1	INFANT MORTALITY RATE MAP	82
22.2.2	POPULATION DENSITY MAP OF THE UNITED STATES	82

22.3	FOLIUM LIBRARY	83
22.4	GEOJSON FILES	83
22.5	EXAMPLE: CHOROPLETH MAP OF IMMIGRATION TO CANADA	83
22.5.1	DATASET OVERVIEW	83
22.6	CREATING THE CHOROPLETH MAP	84
22.6.1	STEP 1: CREATE A WORLD MAP	84
22.6.2	STEP 2: CONVERT THE WORLD MAP INTO A CHOROPLETH MAP	84
22.7	KEY POINTS	84
22.8	SUMMARY	84
23	<u>SUMMARY: ADVANCED VISUALIZATIONS AND GEOSPATIAL DATA</u>	85
23.1	KEY CONCEPTS LEARNED	85
23.1.1	FOLIUM	85
23.1.2	CREATING MAPS	85
23.1.3	MARKERS IN FOLIUM	85
23.1.4	CHOROPLETH MAPS	85
23.1.5	MAPBOX BRIGHT TILESET	86
24	<u>DASHBOARDING OVERVIEW</u>	86
24.1	INTRODUCTION	86
24.2	EXAMPLE: MONITORING DOMESTIC US FLIGHT PERFORMANCE	86
24.3	METHODS OF PRESENTING REPORTS	86
24.3.1	TYPE 1: TABLES AND DOCUMENTS	86
24.3.2	TYPE 2: DASHBOARD FORMAT	87
24.4	WHY DASHBOARDS ARE IMPORTANT	87
25	<u>WEB-BASED DASHBOARDING TOOLS IN PYTHON</u>	87
25.1	DASH	87
25.2	PANEL	88
25.3	VOILA	88
25.4	STREAMLIT	88
25.4.1	MAIN PRINCIPLES OF STREAMLIT	88
25.5	OTHER DASHBOARDING TOOLS	88
25.5.1	BOKEH	88
25.5.2	IPYWIDGETS	89
25.5.3	MATPLOTLIB	89
25.5.4	BOWTIE	89
25.5.5	FLASK	89
25.6	KEY TAKEAWAYS	89
26	<u>INTRODUCTION TO PLOTLY</u>	90

26.1	WHAT IS PLOTLY?	90
26.2	PLOTLY SUB-MODULES	90
26.2.1	PLOTLY GRAPH OBJECTS	90
26.2.2	PLOTLY EXPRESS	90
26.3	CREATING A LINE CHART USING PLOTLY GRAPH OBJECTS	91
26.3.1	IMPORT REQUIRED LIBRARIES	91
26.3.2	GENERATE SAMPLE DATA	91
26.4	CREATING THE FIGURE	91
26.5	UPDATING THE LAYOUT	91
26.6	CREATING THE SAME CHART USING PLOTLY EXPRESS	92
26.6.1	ADVANTAGES OF PLOTLY EXPRESS	92
26.7	AIRLINE REPORTING DATASET	92
26.7.1	OVERVIEW OF THE DATASET	92
26.8	KEY TAKEAWAYS	92
<u>27</u>	<u>INTRODUCTION TO DASH</u>	<u>93</u>
27.1	WHAT IS DASH?	93
27.2	WHY USE DASH?	93
27.3	BUILDING AN APPLICATION WITH DASH	93
27.4	COMPONENTS OF DASH	93
27.4.1	IMPORTING CORE COMPONENTS	94
27.4.2	IMPORTING HTML COMPONENTS	94
27.5	DASH CORE COMPONENTS	94
27.6	DASH HTML COMPONENTS	94
27.7	KEY TAKEAWAYS	94
<u>28</u>	<u>DASH BASICS: HTML AND CORE COMPONENTS</u>	<u>95</u>
28.1	INTRODUCTION	95
28.2	IMPORT REQUIRED PACKAGES	95
28.2.1	PACKAGE DESCRIPTION	95
28.3	LOAD THE DATASET	95
28.4	SAMPLE THE DATA	95
28.4.1	EXPLANATION	95
28.5	CREATE A PIE CHART	96
28.5.1	PARAMETERS	96
28.6	CREATE A DASH APPLICATION	96
28.6.1	EXPLANATION	96
28.7	DEFINE THE APPLICATION LAYOUT	96
28.8	COMPLETE APPLICATION CODE	97
28.9	RUNNING THE APPLICATION	98
28.10	KEY TAKEAWAYS	98
<u>29</u>	<u>MAKING DASHBOARDS INTERACTIVE USING CALLBACKS</u>	<u>98</u>

29.1	UNDERSTANDING CALLBACK FUNCTIONS	98
29.2	CALLBACK FUNCTION WORKFLOW	98
29.3	STRUCTURE OF A CALLBACK FUNCTION	99
29.3.1	STEP 1: CREATE THE FUNCTION	99
29.3.2	STEP 2: DECORATE THE FUNCTION	99
29.4	EXAMPLE: TOP TEN AIRLINE CARRIERS BY YEAR	99
29.5	REQUIRED LIBRARIES	99
29.5.1	NEW IMPORT	99
29.6	LOADING THE DATA	99
29.7	DESIGNING THE APPLICATION LAYOUT	100
29.7.1	ADDING THE DASHBOARD TITLE	100
29.7.2	ADDING AN INPUT COMPONENT	100
29.7.3	ADDING THE GRAPH COMPONENT	100
29.8	CREATING THE CALLBACK	100
29.9	DEFINING THE CALLBACK FUNCTION	101
29.10	RUNNING THE APPLICATION	101
29.11	CALLBACK WITH MULTIPLE INPUTS	101
29.11.1	ADDING ANOTHER INPUT COMPONENT	101
29.11.2	MODIFYING THE CALLBACK DECORATOR	101
29.11.3	UPDATING THE CALLBACK FUNCTION	102
29.12	EXAMPLE OUTPUT	102
29.13	KEY TAKEAWAYS	102
<u>30</u>	<u>SUMMARY: CREATING DASHBOARDS WITH PLOTLY AND DASH</u>	<u>102</u>
30.1	DASH	102
30.2	COMPONENTS OF DASH	103
30.2.1	DASH CORE COMPONENTS	103
30.2.2	DASH HTML COMPONENTS	103
30.3	CALLBACK FUNCTIONS IN DASH	103
30.3.1	THE @APP.CALLBACK DECORATOR	103
30.3.2	WORKING OF CALLBACK FUNCTIONS	103
30.4	KEY TAKEAWAYS	103
<u>31</u>	<u>COMMON ISSUES IN DASH APPLICATIONS</u>	<u>104</u>
31.1	SYNTAX ERROR: INVALID SYNTAX	104
31.1.1	DESCRIPTION	104
31.1.2	EXAMPLE	104
31.1.3	SOLUTION	104
31.2	CALLBACK ERROR UPDATING OUTPUT	104
31.2.1	DESCRIPTION	104
31.2.2	EXAMPLE	104
31.2.3	SOLUTION	105
31.3	GRAPH NOT DISPLAYING	105
31.3.1	DESCRIPTION	105

31.3.2	EXAMPLE	105
31.3.3	SOLUTION	105
31.4	FILE COULD NOT BE LOCATED UNDER THE PROJECT	106
31.4.1	DESCRIPTION	106
31.4.2	SOLUTION	106
31.4.3	RECOMMENDATION	106
31.5	PORT 8050 IS IN USE	106
31.5.1	DESCRIPTION	106
31.5.2	EXAMPLE	106
31.5.3	SOLUTION	106
31.5.4	RECOMMENDATION	106

Introduction to Data Visualization Tools

1 Overview of Data Visualization

1.1 Introduction to Data Visualization

Data visualization is the graphical representation of data and information. It involves creating visual representations that help people understand and analyze data more effectively.

Data visualization can take many forms, including:

- Charts
- Graphs
- Interactive dashboards
- Maps
- Infographics

Visualization transforms raw data into a format that is easier to interpret and analyze.

1.2 Types of Data Visualization

1.2.1 Basic Charts and Graphs

Basic charts and graphs are the simplest forms of data visualization used to represent numerical data.

Common examples include:

- Bar charts
- Line charts
- Pie charts

1.2.2 Interactive Dashboards and Maps

More advanced visualizations include:

- Interactive dashboards
- Geographic maps
- Choropleth maps

These visualizations often provide:

- Real-time information
- Dynamic insights
- Interactive exploration capabilities

1.3 Importance of Data Visualization

Data visualization is important for several reasons.

1.3.1 Simplifies Complex Data

Visualization helps people understand complex datasets that may be difficult to comprehend in raw form.

It helps reveal:

- Patterns
- Trends
- Relationships
- Insights

that may not be immediately visible from raw data alone.

1.3.2 Enhances Communication

Visualization enables organizations and professionals to communicate findings and insights more effectively to stakeholders.

Benefits include:

- Better storytelling with data
- Easier interpretation
- Improved decision-making
- Clearer communication

1.3.3 Helps Identify Trends and Patterns

By examining data over time or across dimensions, visualization helps identify:

- Emerging trends
- Hidden relationships
- Opportunities
- Potential problems

This supports informed and data-driven decision-making.

1.4 Real-World Examples of Data Visualization

1.4.1 COVID-19 Visualization by The New York Times

The New York Times used data visualization to help people understand the impact of the COVID-19 pandemic in the United States.

Examples included:

- Line graphs showing daily reported cases and deaths
- Bar graphs displaying vaccination percentages

These visualizations made complex public health data easier to understand.

1.4.2 Airbnb Smart Pricing Framework

Airbnb uses data visualization within its smart pricing framework to help hosts determine optimal listing prices.

A bar graph is used to display:

- Lead time distribution(how the number of bookings are spread across different lead times)
- Market-level aggregated data(combines data from many listings in the same market/location)

This helps hosts maximize profits.

1.4.3 Spotify Pie

Spotify uses a visualization tool called Spotify Pie to generate customized pie charts based on user listening habits.

The charts display distributions of:

- Genres
- Artists
- Tracks listened to by users

1.4.4 Netflix Insight Tool Dashboard

Netflix uses dashboards containing:

- Choropleth maps
- Line graphs
- Bar plots

These dashboards help the company:

- Understand operations
- Identify improvement areas
- Quickly address issues

1.5 Applications of Data Visualization

1.5.1 Business

Businesses use visualization to analyze:

- Market trends
- Financial performance
- Customer behavior

1.5.2 Healthcare

Healthcare professionals use visualization to:

- Identify patient data patterns
- Develop targeted treatment plans
- Improve healthcare outcomes

1.5.3 Education

Visualization helps educators:

- Analyze student performance
- Inform instructional decisions
- Improve academic outcomes

1.5.4 Government

Governments use visualization to:

- Make informed decisions
- Communicate information to the public

1.5.5 Science and Research

Scientists and researchers use visualization to:

- Analyze complex datasets
- Develop insights
- Share findings effectively

1.5.6 Entertainment Industry

Visualization helps entertainment organizations:

- Determine audience ratings
- Understand viewer preferences
- Identify audience interests

1.6 Best Practices for Data Visualization

Creating effective visualizations requires following best practices to ensure data is accurately represented and clearly communicated.

1.6.1 Choosing the Right Visualization Type

Different types of data require different visualization techniques.

Examples:

- Line charts are ideal for showing trends over time
- Bar charts are ideal for comparing values

Choosing the correct chart type improves communication effectiveness.

1.6.2 Keep Visualizations Simple

Effective visualizations should:

- Be clean and easy to read
- Avoid unnecessary complexity
- Focus on the main message

Too much information can confuse the audience.

1.6.3 Use Clear Labels and Context

Always include:

- Clear axis labels
- Descriptive titles
- Legends explaining colors or symbols

This ensures the audience correctly understands the data.

1.6.4 Minimize Clutter

Avoid:

- Unnecessary graphics
- Excessive labels
- Distracting visual elements

Keep the visualization focused on essential information only.

1.6.5 Consider the Audience

When designing visualizations, consider:

- Who will view the visualization
- What information do they need
- Their level of understanding

Visualizations should be tailored to the audience's needs.

1.7 Good vs Bad Data Visualization

1.7.1 Characteristics of Bad Visualization

Bad visualizations often include:

- Too many variables
- Excessive clutter
- Confusing layouts
- Difficult-to-read information

These issues make the data harder to interpret.

1.7.2 Characteristics of Good Visualization

Effective visualizations typically have:

- Distinguishable and consistent colors
- Clear legends
- Easy-to-read values and percentages
- Simple and focused layouts

1.8 Darkhorse Analytics

Darkhorse Analytics is a company created from a research lab at the University of Alberta in 2008.

The company specializes in:

- Data visualization
- Quantitative consulting
- Geospatial analysis

Their philosophy emphasizes:

- Simplicity
- Clarity
- Visual effectiveness

Their approach follows the principle:

Less is more effective, attractive, and impactful.

1.9 Key Takeaways

Data visualization is the process of presenting data visually through:

- Charts
- Graphs
- Maps
- Dashboards

It helps people:

- Understand data more easily
- Identify patterns and trends
- Communicate insights effectively
- Make informed decisions

Data visualization is widely used in:

- Business
- Healthcare
- Science
- Education
- Government
- Entertainment

Important best practices include:

- Choosing appropriate visualization types
- Using readable colors and fonts
- Labeling clearly
- Minimizing clutter
- Designing for the target audience

2 Types of Plots

2.1 Introduction to Data Visualization

Data visualization represents data using visual formats such as:

- Graphs
- Charts
- Maps

It helps communicate information, trends, and insights effectively in a concise manner. Data visualization also:

- Uncovers patterns
- Identifies trends
- Simplifies complex information
- Improves understanding and interpretation of data

Common types of plots include:

- Line Plot

- Bar Plot
- Scatter Plot
- Box Plot
- Histogram

2.2 Line Plot

A line plot, also called a line chart, displays data as a series of points connected by straight lines.

2.2.1 Uses of Line Plots

Line plots are commonly used for:

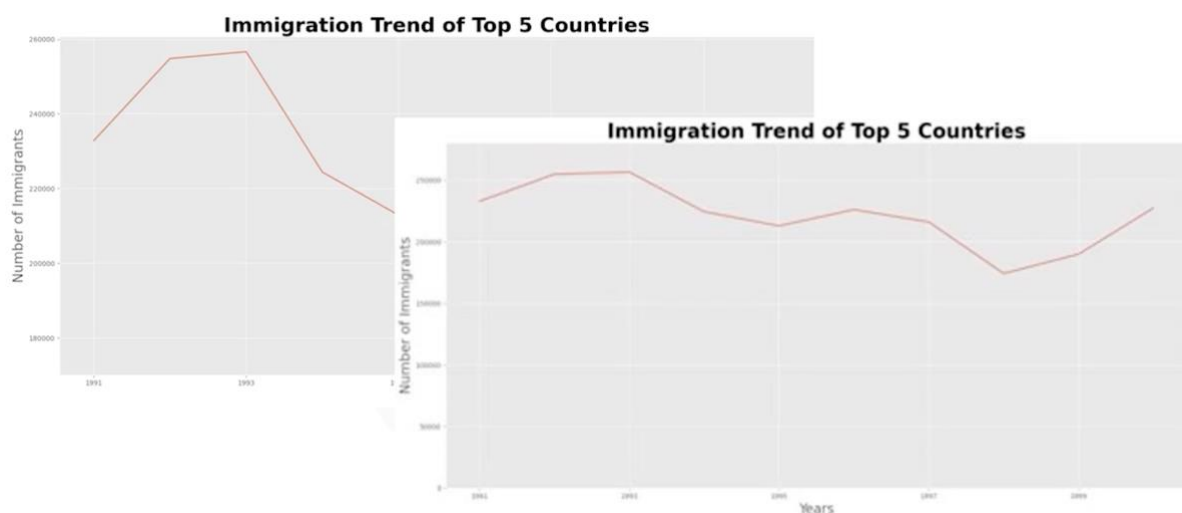
- Displaying trends over time
 - Example: Stock market fluctuations or temperature changes
- Comparing datasets with continuous independent variables
 - Example: Age or time
- Illustrating cause-and-effect relationships
 - Example: Sales revenue changes based on marketing budget
- Visualizing continuous measurements
 - Example: Height measurements over time

2.2.2 Important Considerations

Improper axis scaling can create misleading interpretations.

Example:

- A line plot may appear to show a dramatic decline in immigration trends in 1998
- If the y-axis does not start from zero, the decline may seem exaggerated
- Correcting the scale to start from zero provides a more accurate representation of the trend



2.3 Bar Plot

A bar plot, also known as a bar chart, represents data using rectangular bars where the height or length corresponds to the magnitude of the values.

Bars may be:

- Vertical
- Horizontal

A vertical bar chart is also called a column chart.

2.3.1 Uses of Bar Plots

Bar plots are useful for:

- Comparing categories or groups
- Working with discrete data
 - Example: Comparing product sales revenue
- Showing the contribution of categories to the whole
 - Example: Budget allocation or sales percentages
- Displaying rankings
 - Example: Best-selling books

2.3.2 Advantages of Bar Plots

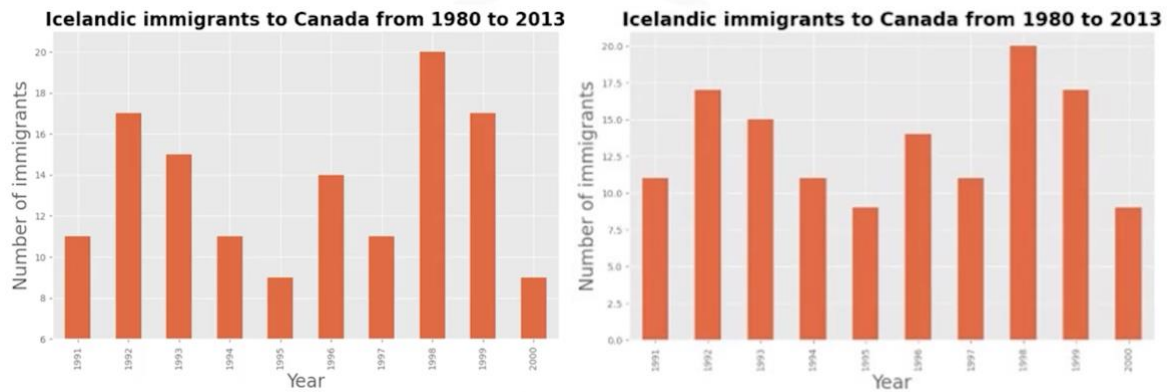
- Simple to understand
- Easy to interpret
- Effective for categorical comparisons

2.3.3 Important Considerations

Improper axis scaling can mislead viewers.

Example:

- If the y-axis starts at 6 instead of 0, differences between values may appear much larger
- Starting the y-axis at zero provides a fair and accurate comparison



2.4 Scatter Plot

A scatter plot displays values for two variables using Cartesian coordinates.

Each point represents:

- One variable on the horizontal axis
- Another variable on the vertical axis

2.4.1 Uses of Scatter Plots

Scatter plots are useful for:

- Examining relationships between two continuous variables
 - Example: Temperature vs. energy consumption
- Investigating patterns or trends
 - Example: House prices vs. house size
- Detecting outliers or unusual observations
 - Example: Abnormal stock behavior or unusual test scores
- Identifying clusters or groups in large datasets
- Exploring complex datasets

2.4.2 Outlier Handling

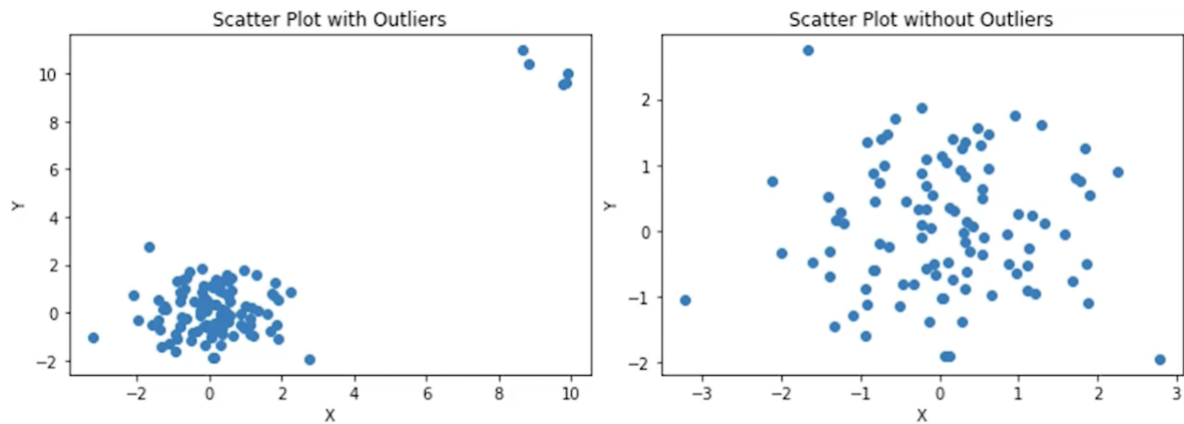
Outliers can significantly impact interpretation.

Example:

- With outliers included, the data may appear scattered
- Removing outliers may reveal hidden clusters more clearly

Proper handling of outliers improves:

- Accuracy
- Clarity
- Meaningful insights



2.5 Box Plot

A box plot, also called a box-and-whisker plot, visualizes the distribution of a dataset along with important statistical measures.

2.5.1 Components of a Box Plot

A box plot consists of:

- A box representing the Interquartile Range (IQR)
- A line inside the box representing the median
- Whiskers extending from the box showing the data range, excluding outliers
- Individual points representing outliers

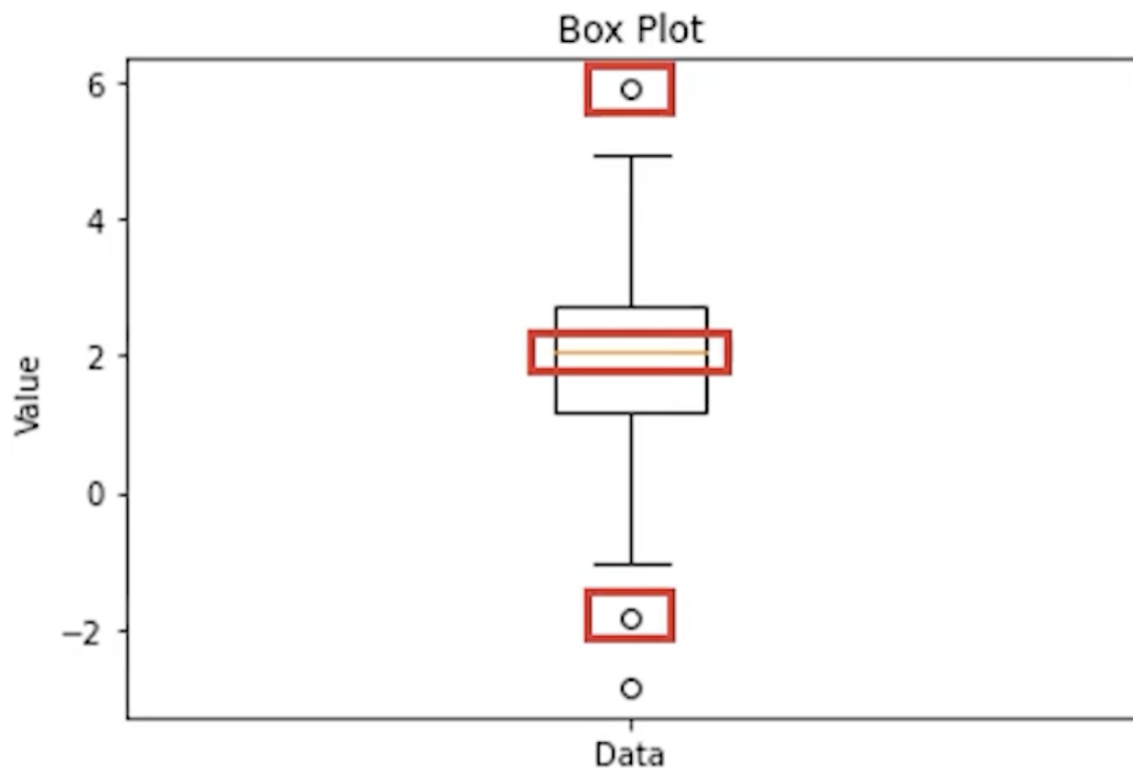
2.5.2 Uses of Box Plots

Box plots are useful for:

- Comparing distributions across groups
 - Example: Employee salaries across departments
- Examining the spread and skewness of data
- Visualizing quartiles and outliers
- Identifying potential outliers
- Presenting summary statistics concisely
 - Median
 - Quartiles
 - Range
- Comparing multiple distributions side by side

2.5.3 Important Considerations

Ignoring or mishandling outliers can distort data interpretation and hide meaningful insights.



2.6 Histogram

A histogram is a graphical representation of data distribution showing frequencies within intervals.

It consists of bars where:

- The height represents the frequency or count of data values in each interval

2.6.1 Uses of Histograms

Histograms help in:

- Understanding data distribution
- Detecting outliers
- Analyzing skewness
- Measuring variability

They can reveal whether the data is:

- Symmetric
- Skewed
- Bimodal

Histograms also help identify:

- Concentrations

- Gaps
- Clusters
- Patterns
- Subgroups

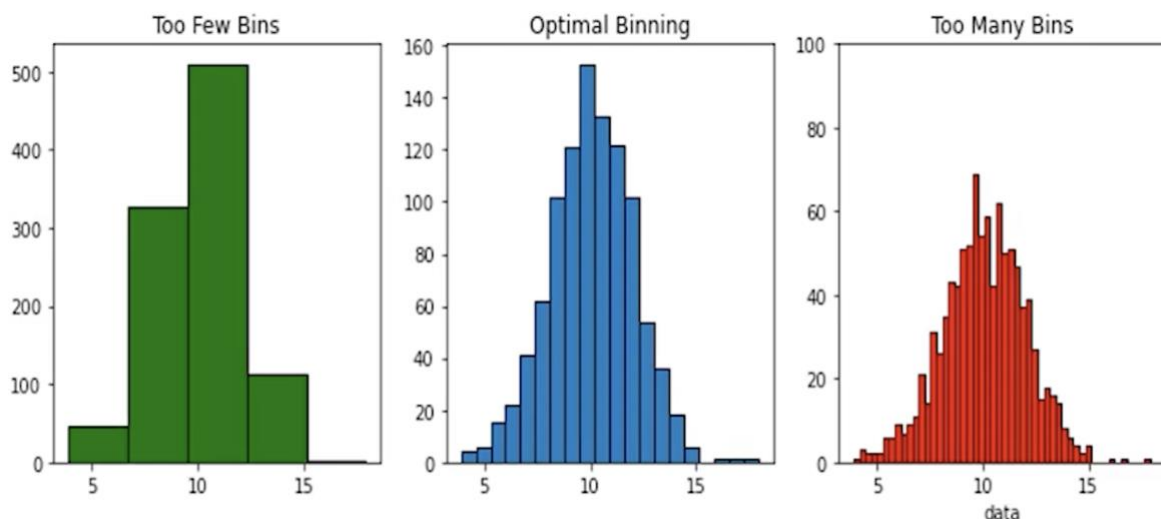
2.6.2 Choosing the Right Number of Bins

Selecting the number of bins is critical.

Example comparisons:

- Too few bins (5 bins)
 - Oversimplifies the distribution
- Optimal bins (20 bins)
 - Provides balanced representation
- Too many bins (50 bins)
 - Overcomplicates the distribution

Improper binning can create misleading visualizations.



2.7 Key Takeaways

- Line plots show trends and changes over time
- Bar plots compare categories or groups
- Scatter plots explore relationships between variables
- Box plots summarize distributions using quartiles and outliers
- Histograms visualize data distribution across intervals

Proper scaling, labeling, outlier handling, and bin selection are essential for creating accurate and meaningful visualizations.

3 Plot Libraries in Python

3.1 Introduction to Plot Libraries

Data visualization is a vital tool for gaining insights and effectively communicating complex information. In Python, several plotting libraries offer distinct features and capabilities for creating various types of visualizations.

Some of the popular plot libraries are:

- Matplotlib
- Pandas
- Seaborn
- Folium
- Plotly
- PyWaffle

Each plot library has its own strengths and use cases. By using these libraries, you can unlock insights from your data and effectively communicate your findings.

3.2 Matplotlib

Matplotlib is a general-purpose plotting library that provides a wide variety of plots and customization options.

It integrates well with other libraries and frameworks such as:

- NumPy
- Pandas
- Seaborn
- Plotly

This interoperability (the ability of different libraries to work together) allows you to combine the strengths of multiple libraries and create sophisticated and interactive visualizations.

3.2.1 Features of Matplotlib

Matplotlib allows you to create:

- Line plots
- Scatter plots
- Bar charts
- Histograms
- Pie charts
- Box plots
- Heat maps

It also provides extensive customization options, including:

- Colors
- Line styles
- Marker styles
- Axis labels
- Titles
- Legends

3.2.2 Advantages of Matplotlib

- Highly versatile
- Extensive customization capabilities
- Strong integration with other libraries
- Large community support
- Reliable for data visualization tasks in Python

3.3 Pandas Plotting Library

Pandas is primarily used for data manipulation, but it also offers built-in plotting capabilities.

Pandas plotting functions are built on top of Matplotlib, which means you can combine Pandas data manipulation capabilities with the power and flexibility of Matplotlib.

3.3.1 Features of Pandas Plotting

Pandas integrates seamlessly with Pandas data structures, making it easy to create:

- Line plots
- Scatter plots
- Bar charts
- Histograms
- Pie charts

directly from DataFrames.

3.3.2 Advantages of Pandas Visualization

- Easy integration with DataFrames
- Quick visualization for exploratory data analysis (EDA)
- Simple and efficient plotting functionality

3.4 Seaborn

Seaborn is built upon the functionality of Matplotlib and is widely used for specialized statistical visualizations.

It provides stylish and specialized plot types such as:

- Categorical plots
- Count plots

- Heat maps
- Violin plots
- Scatter plots
- Bar plots

3.4.1 Features of Seaborn

Seaborn offers:

- Attractive default aesthetics (visual appearance of plots)
- Various color palettes
- Built-in themes
- Grid layout functions for combining multiple plots

These features help compare multiple variables or subgroups effectively.

3.4.2 Integration with Pandas

Seaborn integrates directly with Pandas and can plot data from:

- Pandas DataFrames
- Pandas Series

3.4.3 Advantages of Seaborn

- Specialized statistical visualizations
- Better default aesthetics
- Easy-to-use interface
- Strong integration with Pandas
- Built-in themes and color palettes

Seaborn is a popular choice among data scientists and analysts for exploring and communicating insights from data.

3.5 Folium

Folium is a powerful library for geospatial data visualization (visualizing location-based or map-related data).

It allows you to create interactive and customizable maps.

3.5.1 Types of Maps in Folium

Folium supports:

- Choropleth maps (maps using colors to represent data values across regions)
- Point maps
- Heat maps

3.5.2 Features of Folium

- Interactive map visualizations
- Customizable mapping tools
- Integration with Pandas and NumPy
- Useful for geospatial analysis

Folium is widely used for geospatial data visualization and analysis in Python.

3.6 Plotly

Plotly is an interactive and dynamic visualization library.

It is widely used for creating interactive plots and dashboards.

3.6.1 Types of Visualizations in Plotly

Plotly supports:

- Line plots
- Scatter plots
- Bar charts
- Pie charts
- 3D plots
- Choropleth maps

3.6.2 Features of Plotly

- Highly interactive visualizations
- Dashboard creation using the Plotly Dash framework
- Web-based rendering and visualization
- Easy sharing and embedding in web applications

Since Plotly is web-based, plots can be rendered and viewed directly in web browsers. This makes it convenient for:

- Sharing visualizations online
- Embedding plots into dashboards or web applications
- Collaboration with others

3.7 PyWaffle

PyWaffle is a simple yet effective library used for visualizing categorical data using Waffle charts.

3.7.1 Types of Visualizations in PyWaffle

PyWaffle can create:

- Waffle charts
- Square pie charts
- Donut charts

These visualizations provide a unique way to represent proportions using squares or rectangles.

3.7.2 Advantages of PyWaffle

- Simple and easy-to-use
- Effective for proportional representation(Showing how much each category contributes compared to the total)
- Unique visualization style for categorical data

3.8 Summary of Plot Libraries

3.8.1 Matplotlib

- General-purpose plotting library
- Wide range of plotting capabilities
- Highly customizable

3.8.2 Pandas

- Integrated plotting functionalities for data analysis
- Built on top of Matplotlib
- Best suited for quick exploratory analysis

3.8.3 Seaborn

- Specialized statistical visualization library
- Attractive default aesthetics and color palettes
- Strong integration with Pandas

3.8.4 Folium

- Library for interactive and customizable maps
- Excellent for geospatial data visualization

3.8.5 Plotly

- Interactive and dynamic visualization library
- Supports dashboards and web-based visualizations
- Wide range of plot types

3.8.6 PyWaffle

- Used for proportional representation using squares or rectangles
- Best suited for Waffle charts and categorical data visualization

4 Introduction to Matplotlib

4.1 What is Matplotlib?

Matplotlib is one of the most widely used data visualization libraries in Python. It was created by John Hunter, an American neurobiologist.

4.2 History and Origin

- John Hunter was part of a research team analyzing electrocorticography (ECoG) signals using proprietary software. However, the team had only one software license, so researchers had to take turns using it.
- To overcome this limitation, John Hunter decided to replace the proprietary software with a MATLAB-based version that could be used and extended by multiple researchers. As a result, Matplotlib was initially developed as an EEG(Electroencephalography) and ECoG visualization tool.
- Just like MATLAB, Matplotlib was designed with a scripting interface for the quick and easy generation of graphics and plots.

4.3 Uses of Matplotlib

Matplotlib is widely used for:

- Data visualization
- Exploratory data analysis
- Creating plots and charts
- Scientific visualization
- Statistical plotting
- Interactive graphics generation

It supports the creation of:

- Histograms
- Line plots
- Scatter plots
- Bar charts
- Pie charts
- Subplots
- Annotations and labels

4.4 Matplotlib Architecture

Matplotlib architecture is composed of three main layers:

- Backend Layer
- Artist Layer
- Scripting Layer

4.4.1 Backend Layer

The backend layer contains three built-in abstract interface classes:

4.4.1.1 FigureCanvas

FigureCanvas defines and encompasses the area on which the figure is drawn.

4.4.1.2 Renderer

The Renderer class knows how to draw on the FigureCanvas.

4.4.1.3 Event

The Event handler manages user interactions such as:

- Keyboard strokes
- Mouse clicks

4.4.2 Artist Layer

The artist is the object responsible for taking the renderer and putting ink on the canvas.

Everything visible in a Matplotlib figure is an artist instance, including:

- Titles
- Lines
- Tick labels
- Images
- Shapes
- Text

4.4.2.1 Types of Artist Objects

There are two main types of artist objects.

4.4.2.1.1 Primitive Artists

Primitive artists include:

- Line2D
- Rectangle
- Circle
- Text

4.4.2.1.2 Composite Artists

Composite artists include:

- Axis
- Tick
- Axes
- Figure

The top-level Matplotlib object that manages all elements in a graphic is the Figure artist.

The most important composite artist is the Axis because it contains most of the Matplotlib API plotting methods, including methods used to:

- Create ticks
- Manipulate axis lines
- Add grids
- Control plot backgrounds

Each composite artist may contain both primitive artists and other composite artists.

For example, a Figure artist can contain:

- Axis artist
- Rectangle artist
- Text artist

4.4.3 Scripting Layer (Pyplot)

The scripting layer is designed for everyday visualization tasks and quick graphics generation.

Matplotlib's scripting layer is the `matplotlib.pyplot` interface, commonly called Pyplot.

Pyplot automatically:

- Defines a Canvas
- Creates a Figure artist instance
- Connects the Canvas and Figure

The scripting layer is simpler and easier to use compared to directly working with the backend and artist layers.

Scientists and data analysts commonly use Pyplot for:

- Data visualization
- Exploratory analysis
- Quick plotting tasks

4.5 Generating a Histogram Using Pyplot

A histogram of 10,000 random numbers can be generated using Pyplot and NumPy.

4.5.1 Example Code

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.randn(10000)

plt.hist(x, 100)

plt.show()
```

4.6 Explanation of the Histogram Code

4.6.1 Importing Libraries

```
import matplotlib.pyplot as plt
import numpy as np
```

- matplotlib.pyplot provides plotting functions
- NumPy is used for numerical operations and random number generation

4.6.2 Generating Random Numbers

```
x = np.random.randn(10000)
```

- random.randn() generates random floating-point numbers
- 10000 means an array of 10,000 random numbers is created

4.6.3 Creating the Histogram

```
plt.hist(x, 100)
```

- x contains the random numbers
- 100 specifies the number of bins (groups/categories)

The hist() function creates rectangle artists for each histogram bar and adds them to the Axes container.

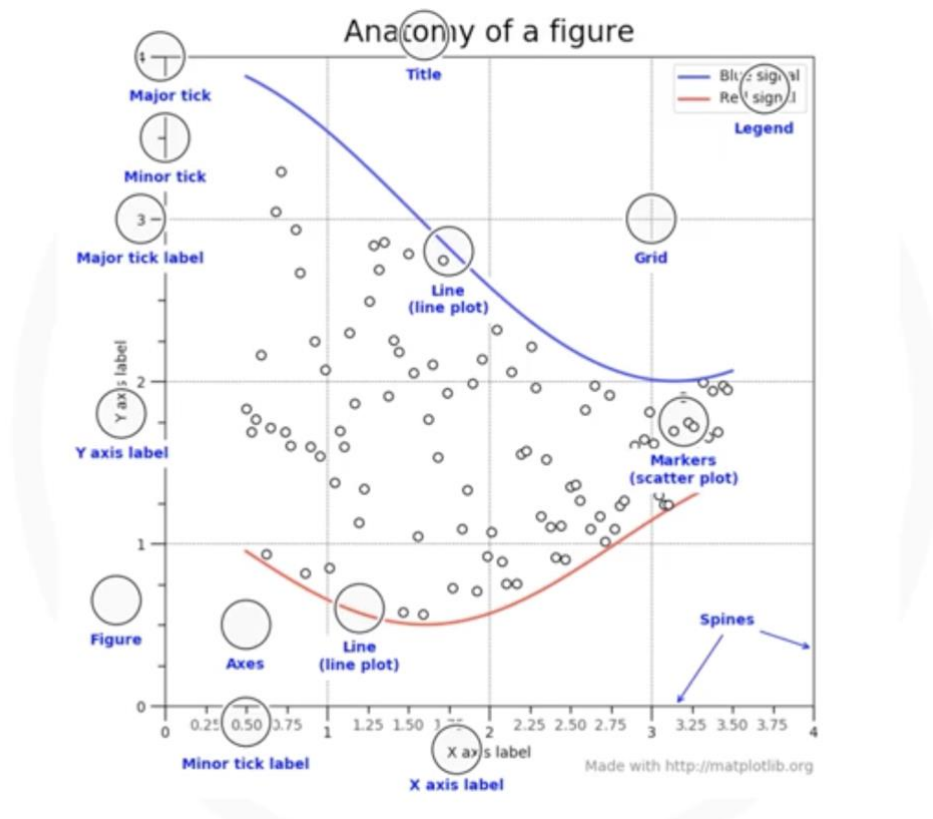
4.6.4 Displaying the Plot

```
plt.show()
```

This displays the generated histogram.

4.7 Anatomy of a Plot

The anatomy of a plot refers to the different components and elements that make up a visual representation of data.



4.7.1 Main Components of a Plot

4.7.1.1 Figure (Canvas)

The Figure acts as the main window or canvas containing plots or subplots.

4.7.1.2 Axes

Axes represent individual plots within a Figure.

To create multiple plots in one Figure, multiple Axes objects are required.

4.7.1.3 Axis

The Axis provides:

- Scales
- Tick marks
- Coordinate references

4.7.1.4 Data Points

The plotted data appears as:

- Points
- Lines

- Markers
- Bars

4.7.2 Additional Plot Components

4.7.2.1 Title

A title provides a summary or explanation of the plot.

4.7.2.2 Axis Labels

Labels describe the data plotted on each axis.

4.7.2.3 Legend

A legend explains the meaning of different data series or plot elements.

4.7.2.4 Grid

A grid helps:

- Visually align data points
- Improve readability
- Estimate values more easily

4.7.2.5 Annotations

Annotations provide additional explanations or information about:

- Specific data points
- Particular regions of the plot

4.7.3 Plot Customization

Matplotlib allows customization of:

- Symbols
- Colors
- Marker styles
- Line styles

4.8 Summary

- Matplotlib is one of the most widely used Python data visualization libraries.
- It was initially developed as an EEG and ECoG visualization tool.
- Matplotlib architecture consists of:
 - Backend Layer
 - Artist Layer
 - Scripting Layer

- The Pyplot interface simplifies plotting and visualization tasks.
- The anatomy of a plot includes components such as:
 - Figure
 - Axes
 - Axis
 - Titles
 - Labels
 - Legends
 - Grids
 - Annotations

5 Basic Plotting with Matplotlib

5.1 Introduction to Matplotlib

Matplotlib is a well-established data visualization library in Python used to create different types of plots and charts. It can be integrated into multiple environments, such as:

- Python scripts
- Python and IPython shells
- Web application servers
- Graphical User Interface (GUI) toolkits
- Jupyter Notebook

Matplotlib is widely used for creating visualizations such as:

- Line plots
- Histograms
- Bar charts
- Box plots
- Many other conventional visualization tools

5.2 Jupyter Notebook and Matplotlib

Jupyter Notebook is an open-source web application that allows users to create and share documents containing:

- Live code
- Visualizations
- Explanatory text

Jupyter Notebook has specialized support for Matplotlib. To start using Matplotlib in Jupyter Notebook, simply import Matplotlib and begin plotting.

5.3 Using the Scripting Interface

Matplotlib visualizations can be created using the scripting interface.

The scripting interface is commonly imported as:

```
import matplotlib.pyplot as plt
```

One important advantage of this interface is that almost all conventional visualization tools can be created using a single function:

```
plt.plot()
```

This function can be used to create different types of plots depending on the provided parameters.

5.4 Creating a Basic Plot

A simple plot can be created by plotting a circular mark at position (5,5):

```
import matplotlib.pyplot as plt
plt.plot(5, 5, 'o')
plt.show()
```

In Jupyter Notebook, the plot is generated directly inside the browser instead of opening in a separate pop-up window.

5.5 Using the %matplotlib inline Magic Function

If plots are generated in a pop-up window instead of inside the notebook, the following magic function can be used:

```
%matplotlib inline
```

This command forces plots to render directly inside the Jupyter Notebook.

5.5.1 Important Note

One limitation of the inline backend is:

- Once a figure is rendered, it cannot be modified.
- After rendering, you cannot add:
 - Figure titles
 - Axis labels
 - Other modifications

To include titles or labels, they must be added before calling:

```
plt.show()
```

Example:

```
import matplotlib.pyplot as plt
```



```
plt.plot(5, 5, 'o')
plt.title("Sample Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")

plt.show()
```

5.6 Notebook Backend

Another useful backend is the notebook backend.

The notebook backend overcomes the limitation of the inline backend.

5.6.1 How It Works

- If an active figure already exists, Matplotlib applies new plotting commands to the same figure.
- If no active figure exists, Matplotlib creates a new figure automatically.

Example:

```
plt.plot(5, 5, 'o')
```

If no figure exists:

- A new figure is created
- A circular mark is added at position (5,5)

5.6.2 Advantage of Notebook Backend

With the notebook backend, titles and labels can be added even after the plot has already been rendered, without regenerating the figure.

5.7 Matplotlib Integration with Pandas

Pandas has a built-in implementation of Matplotlib.

Because of this integration, plotting in Pandas is very simple.

Plots can be created directly from:

- Pandas Series
- Pandas DataFrames

using the `plot()` function.

5.8 Creating a Line Plot Using Pandas

Suppose there is a DataFrame named:

```
India_China_df
```

This DataFrame contains the number of immigrants to Canada from:

- India
- China

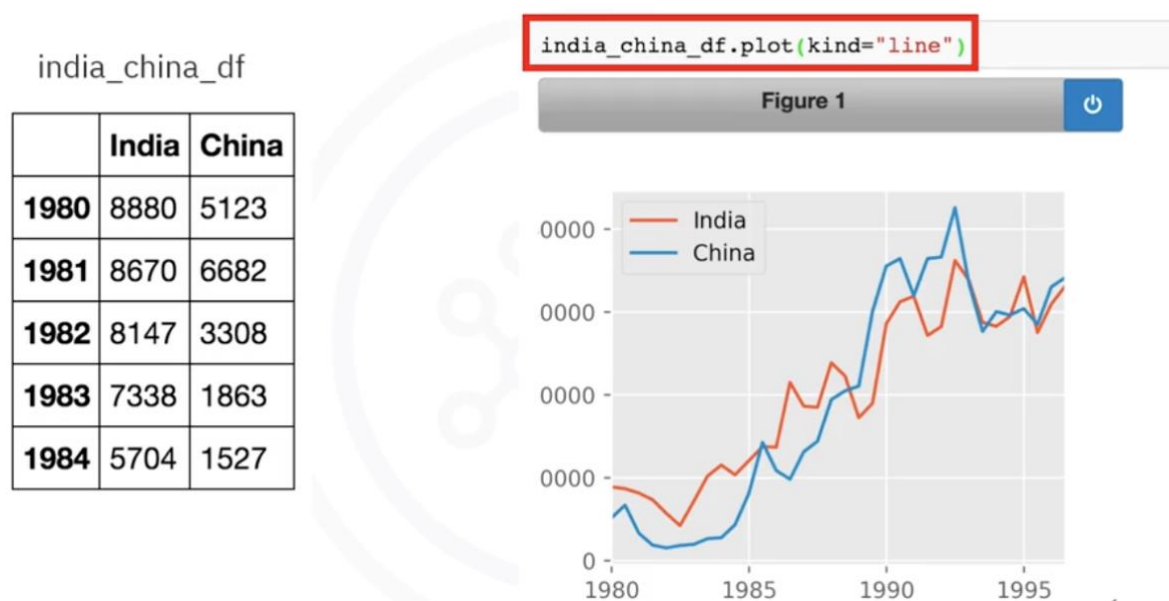
for the years:

- 1980 to 1996

A line plot can be created using:

```
India_China_df.plot(kind='line')
```

This generates a line plot of the data in the DataFrame.



5.9 Creating a Histogram Using Pandas

To plot a histogram of the India column:

```
India_China_df['India'].plot(kind='hist')
```

This creates a histogram showing the number of Indian immigrants to Canada from 1980 to 1996.

5.10 Key Points

- Matplotlib is a powerful and widely used data visualization library.
- It can be integrated into multiple environments.
- Jupyter Notebook supports Matplotlib directly.
- The `plt.plot()` function can create many different visualization types.
- `%matplotlib inline` displays plots inside Jupyter Notebook.
- The notebook backend allows modification of plots after rendering.

- Pandas includes built-in support for Matplotlib.
- Plotting in Pandas is as simple as calling the `plot()` function on a Series or DataFrame.

6 Dataset on Immigration to Canada

6.1 Overview

The dataset used in this course is compiled by the Population Division of the United Nations. It contains immigration data pertaining to 45 countries. For each country, the dataset reports the total number of immigrants from countries all around the world.

In addition to immigration numbers, the dataset also contains metadata (additional information) about the immigrants' countries of origin, such as:

- Continent
- Region
- Development status (developing or developed region)

In this course, the primary focus is on immigration to Canada, also known as the “Great White North.”

6.2 Structure of the Dataset

The UN immigration dataset is provided in the form of an Excel spreadsheet file.

6.2.1 Important Parts of the Dataset

- The first 20 rows contain textual information about the United Nations department and other metadata.
- Row 21 contains the column labels (column names).
- Each row after that represents a country.

Each country row includes:

- Country name
- Continent
- Region
- Development status
- Immigration data from 1980 to 2013

The yearly columns show the total number of immigrants from a specific country to Canada for each year.

6.3 Using Pandas for Data Analysis

Throughout this course, pandas will be used for data analysis before creating visualizations.

To begin creating plots and charts for exploratory analysis or presentation purposes, the dataset must first be imported into a pandas DataFrame.

6.4 Required Libraries

The following libraries are required:

- pandas
- openpyxl

The openpyxl library is required to read Excel spreadsheet files.

6.4.1 Importing Libraries

```
import pandas as pd
import openpyxl
```

6.5 Reading the Excel File

The pandas read_excel() function is used to import the dataset into a DataFrame.

Since the first 20 rows contain unnecessary text, they are skipped when reading the file.

6.5.1 Reading Data into a DataFrame

```
df = pd.read_excel('Canada.xlsx', skiprows=20)
```

6.5.2 Displaying the First Five Rows

The head() function is used to confirm that the data has been imported correctly.

```
df.head()
```

6.6 Processing the DataFrame

After importing the dataset, the DataFrame is processed to make analysis and plotting easier.

6.6.1 Setting Country Name as the Index

The country name is used as the index of each row. This makes querying (searching and accessing) specific countries easier.

```
df.set_index('Country', inplace=True)
```

6.7 Adding Total Immigration Column

An additional column is created to represent the total immigration for each country from 1980 to 2013.

Examples:

- Afghanistan → 58,639
- Albania → 15,699

6.7.1 Creating Total Column

```
df['Total'] = df.sum(axis=1)
```

6.8 Naming the DataFrame

The processed DataFrame is named `df_canada`.

```
df_canada = df
```

6.9 Summary

The Population Division of the United Nations compiled immigration data related to migration to Canada.

The dataset contains:

- Immigration data from 1980 to 2013
- Country metadata
- Regional and developmental information

Before creating visualizations, the dataset must be:

- Imported into a pandas DataFrame
- Cleaned and processed
- Structured for easier querying and plotting

Pandas provides the necessary tools to prepare data for various visualizations and exploratory data analysis (EDA).

7 Line Plots

7.1 Introduction to Line Plots

A line plot is a plot that displays information as a series of data points connected by straight lines. It is one of the most basic types of charts and is commonly used in many fields, not just data science.

7.2 When to Use a Line Plot

Line plots are useful for visualizing trends and changes over time, making them a popular choice for time-series data.

Examples include:

- Changes in stock prices
- Website traffic analysis
- Temperature fluctuations

Line plots can also be used to:

- Show relationships between two variables
- Compare multiple data series on one chart
- Highlight sudden changes or anomalies in data

7.3 Example: Immigration from Haiti to Canada

Using the immigration dataset, a line plot can be generated to visualize the trend of immigrants from Haiti to Canada.

The line plot shows a noticeable spike in immigration in 2010. By researching major events in Haiti during 2010, one can identify the tragic earthquake that occurred that year. Therefore, the increase in immigration to Canada was mainly due to the effects of that earthquake.

This example demonstrates how line plots help identify trends, sudden changes, and anomalies in data.

7.4 Understanding the Dataset

The dataset contains immigration information for different countries.

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	2006	2007	2008	2009	2010	2011	2012	2013	Total
Country																					
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	3009	2652	2111	1746	1758	2203	2635	2004	58639
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	856	702	560	716	561	539	620	603	15699
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	4807	3623	4005	5393	4752	4325	3774	4331	69439
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	1	0	0	0	0	0	0	0	6
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	1	1	0	0	0	0	1	1	15

7.4.1 Dataset Structure

- Each row represents a country
- The dataset includes information about:
 - Geographic region
 - Whether the country is developing or developed
- Each row also contains numerical figures representing annual immigration from that country to Canada from 1980 to 2013

7.5 DataFrame Processing

To make data analysis easier:

- The country name is set as the index of each row
- An additional column is added representing the total immigration for each country from 1980 to 2013

Examples:

- Afghanistan → 58,639
- Albania → 15,699

The processed DataFrame is named:

```
df_canada
```

7.6 Generating a Line Plot

To generate a line plot for immigration from Haiti to Canada:

7.6.1 Step 1: Import Required Libraries

```
import matplotlib.pyplot as plt
```

7.6.2 Step 2: Generate the Line Plot

Call the `plot()` function on the row corresponding to Haiti and specify:

```
kind='line'
```

Example:

```
df_canada.loc['Haiti', years].plot(kind='line')
```

7.6.3 Step 3: Add Title and Axis Labels

```
plt.title('Immigration from Haiti to Canada')  
plt.xlabel('Years')  
plt.ylabel('Number of Immigrants')
```

7.6.4 Step 4: Display the Plot

```
plt.show()
```

7.7 Important Note

The line plot was generated using the matplotlib magic function with the inline backend:

```
%matplotlib inline
```

This allows plots to be displayed directly within Jupyter Notebook cells.

7.8 Summary

- A line plot displays data points connected by straight-line segments
- It is one of the most basic and widely used chart types
- Line plots are especially useful for:
 - Visualizing trends over time
 - Comparing multiple datasets
 - Detecting sudden changes or anomalies
- In pandas, a line plot can be generated by setting:

`kind='line'` inside the `plot()` function.

8 Summary: Introduction to Data Visualization Tools

8.1 Overview of Data Visualization

Data visualization is the process of presenting data in a visual format, such as charts, graphs, and maps, to help people understand and analyze data easily.

Data visualization has diverse use cases, such as in:

- Business
- Science
- Healthcare
- Finance

It is important to follow best practices while creating visualizations, including:

- Selecting appropriate visualizations for the data being presented
- Choosing colors and fonts that are easy to read and interpret
- Minimizing clutter to improve clarity and readability

8.2 Common Types of Data Visualizations

8.2.1 Line Plots

A line plot is a plot in the form of a series of data points connected by straight line segments.

Line plots:

- Capture trends and changes over time
- Allow us to see patterns and fluctuations
- Are one of the most basic types of charts
- Are commonly used in many fields

You can generate a line plot by assigning "line" to the kind parameter in the plot() function.

Example:

```
df.plot(kind='line')
```

8.2.2 Bar Plots

Bar plots:

- Compare categories or groups
- Provide a visual comparison of values between different groups

8.2.3 Scatter Plots

Scatter plots:

- Explore relationships between variables

- Help identify correlations or trends between variables

8.2.4 Box Plots

Box plots:

- Display the distribution of data
- Show the median, quartiles, and outliers

8.2.5 Histograms

Histograms:

- Illustrate the distribution of data within specific intervals
- Help us understand the shape and concentration of data

8.3 Data Visualization Libraries

8.3.1 Matplotlib

Matplotlib is one of the most widely used data visualization libraries in Python.

Matplotlib:

- Offers a wide range of plotting capabilities
- Is a well-established data visualization library
- Can be integrated into different environments

Matplotlib was initially developed as an EEG/ECoG visualization tool.

8.3.2 Matplotlib Architecture

Matplotlib's architecture is composed of three main layers:

- Backend layer
- Artist layer
- Scripting layer

8.3.3 Anatomy of a Plot

The anatomy of a plot refers to the different components and elements that make up a visual representation of data.

You can easily include labels and titles in your plots using `plt`.

Example:

```
import matplotlib.pyplot as plt

plt.title("Sample Plot")
```

```
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")
```

8.3.4 Pandas

Pandas is a plotting library that provides integrated plotting functionalities for data analysis.

In order to start creating different types of plots, you need to import the data into a Pandas DataFrame.

Example:

```
import pandas as pd  
  
df = pd.read_csv("data.csv")
```

8.3.5 Seaborn

Seaborn is a specialized library for statistical visualizations.

Seaborn provides:

- Attractive default aesthetics
- Built-in color palettes
- Statistical visualization capabilities

8.3.6 Folium

Folium is a Python library that allows you to create:

- Interactive maps
- Customizable maps

8.3.7 Plotly

Plotly is an interactive and dynamic library for data visualization.

Plotly:

- Supports a wide range of plot types
- Provides interactive features for exploration and analysis

8.3.8 PyWaffle

PyWaffle enables you to visualize proportional representation using:

- Squares
- Rectangles

8.4 Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to:

- Create documents
- Share documents
- Combine code, visualizations, and text in one environment

8.5 Matplotlib Backends

Matplotlib has several backends available.

(Backends are components responsible for rendering plots in different environments, such as interactive windows, web applications, or image files.)

Module 2: Basic and Specialized Visualization Tools

9 Area Plots

An Area Plot, also known as an Area Chart or Area Graph, displays the magnitude and proportion of multiple variables over a continuous axis, typically representing time or another ordered dimension.

It is similar to a Line Plot, but the area below the line is filled with color to emphasize the cumulative magnitude of the variables.

Area plots are commonly used when comparing two or more quantities over time.

9.1 Understanding the Dataset

The dataset contains immigration data to Canada from 1980 to 2013.

Each row in the dataset represents a country and contains:

- Metadata about the country
 - Geographic location
 - Development status
- Numerical data showing annual immigration from that country to Canada from 1980 to 2013

To make data retrieval easier:

- The country name is set as the index of each row
- An additional column named Total is added to represent the cumulative sum of annual immigration from 1980 to 2013

Example:

- Afghanistan → 58,639 total immigrants
- Albania → 15,699 total immigrants

The processed DataFrame is named:

`df_canada`

9.2 Finding the Top Immigration Countries

To identify the countries with the highest immigration to Canada:

- The DataFrame is sorted in descending order using the cumulative Total column
- The `sort_values()` function is used for sorting

The top five countries with the highest immigration are:

- India
- China
- United Kingdom
- Philippines
- Pakistan

9.3 Preparing Data for Area Plots

Before generating the Area Plot, the data must be prepared properly.

9.3.1 Creating a DataFrame for the Top Five Countries

A new DataFrame is created containing only the top five countries.

The Total column is excluded because only yearly immigration data is needed.

9.3.2 Why Transpose the DataFrame?

Matplotlib plots DataFrame indices on the horizontal axis. The key idea is that **Matplotlib uses the DataFrame index as the x-axis by default**.

If the DataFrame is used directly:

- Countries appear on the horizontal axis
- Years do not appear correctly

To correct this:

- The DataFrame is transposed using the transpose method `.transpose()` or `.T`

This changes the structure so that:

- Years become the indices (horizontal axis)
- Countries become columns

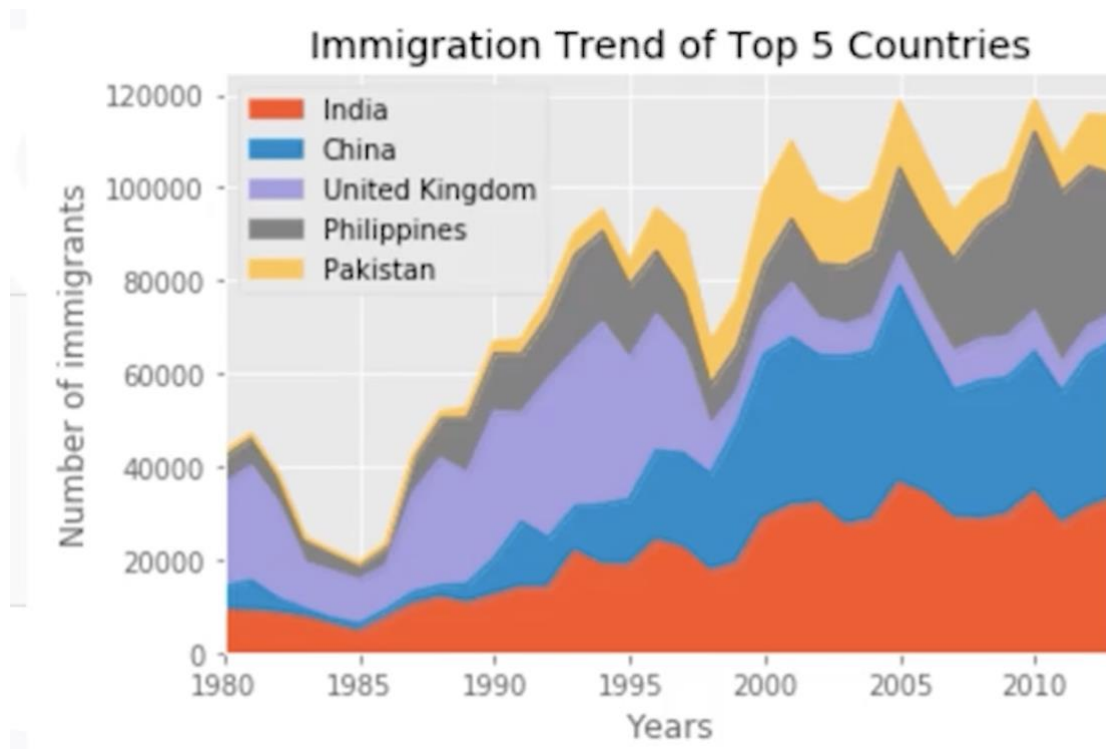
The resulting DataFrame contains:

- Five columns representing the top five countries
- Years from 1980 to 2013 as indices

9.4 Generating an Area Plot Using Matplotlib

To create the Area Plot:

- Import Matplotlib
- Call the plot() function on the DataFrame
- Set kind='area'
- Add title and axis labels
- Display the figure using show()



9.5 Example Code

```
import matplotlib as mpl
import matplotlib.pyplot as plt

# Sort data by total immigration
df_canada.sort_values(
    ['Total'],
    ascending=False,
    axis=0,
    inplace=True
)
```

```
# Create a dataframe of the top 5 countries
df_top5 = df_canada.head()

# Keep only years and transpose the dataframe
df_top5 = df_top5[years].transpose()

# Generate area plot
df_top5.plot(kind='area')

# Add title and labels
plt.title('Immigration Trend of Top 5 Countries')
plt.xlabel('Years')
plt.ylabel('Number of Immigrants')

# Display plot
plt.show()
```

9.6 Understanding the Area Plot

The Area Plot visualizes the immigration trends of the five countries with the highest immigration to Canada from 1980 to 2013.

The filled areas help emphasize:

1. Cumulative trends
2. Relative proportions between countries
3. Changes over time

9.7 Applications of Area Plots

Area plots are particularly effective for displaying cumulative data, such as:

- Tracking stock market performance
- Visualizing population demographics
- Showing resource distribution across sectors
- Comparing trends over time

9.8 Summary

An Area Plot:

- Depicts cumulative totals using numbers or percentages over time
- Is similar to a line plot but uses filled colors beneath the lines
- Helps visualize relationships and proportions among multiple variables

The process of creating an Area Plot involves:

- Importing Matplotlib
- Calling the plot() function on a DataFrame
- Setting the kind parameter to 'area.'

Area plots provide a visually appealing and intuitive way to compare multiple variables in a single chart.

10 Histograms

10.1 What is a Histogram?

A histogram is a way of representing the frequency distribution of a numeric data set.

A histogram works by:

- Partitioning the spread of the numeric data into bins
- Assigning each data point in the data set to a bin
- Counting the number of data points assigned to each bin

The vertical axis represents the frequency (the number of data points in each bin).

10.2 Understanding Histogram Bins

Consider a numeric data set whose values range from 0 to 34,129.

The first step in creating a histogram is partitioning the horizontal axis into bins of equal width. For example, the data can be divided into 10 bins.

Then the histogram is constructed by counting how many data points fall within the limits of each bin.

For example:

1. If 175 data points have values between 0 and 3,413, then a bar with height 175 is drawn for that bin.
2. The same process is repeated for all remaining bins.
3. If no data points fall into a particular bin, then the corresponding bar has a height of zero.

10.3 Preparing the Data for Visualization

To create a histogram effectively, the data frame is processed so that:

- The country name becomes the index of each row
- An additional column named `total` is added

The `total` column represents the cumulative sum of annual immigration from each country from 1980 to 2013.

Examples:

- Afghanistan → 58,639 total immigrants
- Albania → 15,699 total immigrants

The processed data frame is named:

```
df_canada
```

10.4 Visualizing Immigration Distribution for 2013

To visualize the distribution of immigrants to Canada in the year 2013, a histogram can be generated using the data in column 2013.

10.5 Creating a Histogram Using Matplotlib

First, import Matplotlib and its scripting interface:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

Then call the plot function on the data in column 2013 and specify:

```
kind='hist'
```

Example:

```
df_canada['2013'].plot(kind='hist')

plt.title('Histogram of Immigration to Canada in 2013')
plt.xlabel('Number of Immigrants')
plt.ylabel('Number of Countries')

plt.show()
```

This generates a histogram that depicts the distribution of immigration to Canada in 2013.

10.6 Issue with Default Histogram Bins

One issue with the default histogram is that the bins may not align properly with the tick marks on the horizontal axis.

This can make the histogram difficult to read.

10.7 Improving Histogram Alignment Using NumPy

One solution is to use the histogram function from the NumPy library.

First, import NumPy:

```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
```

Then use NumPy's histogram function:

```
count, bin_edges = np.histogram(df_canada['2013'])
```


This function:

- Partitions the data into 10 bins of equal width (default behavior)
- Computes the number of data points in each bin
- Returns:
 - count → frequency of each bin
 - bin_edges → edges (limits) of the bins

Now pass the bin edges into the plotting function:

```
df_canada['2013'].plot(kind='hist', xticks=bin_edges)

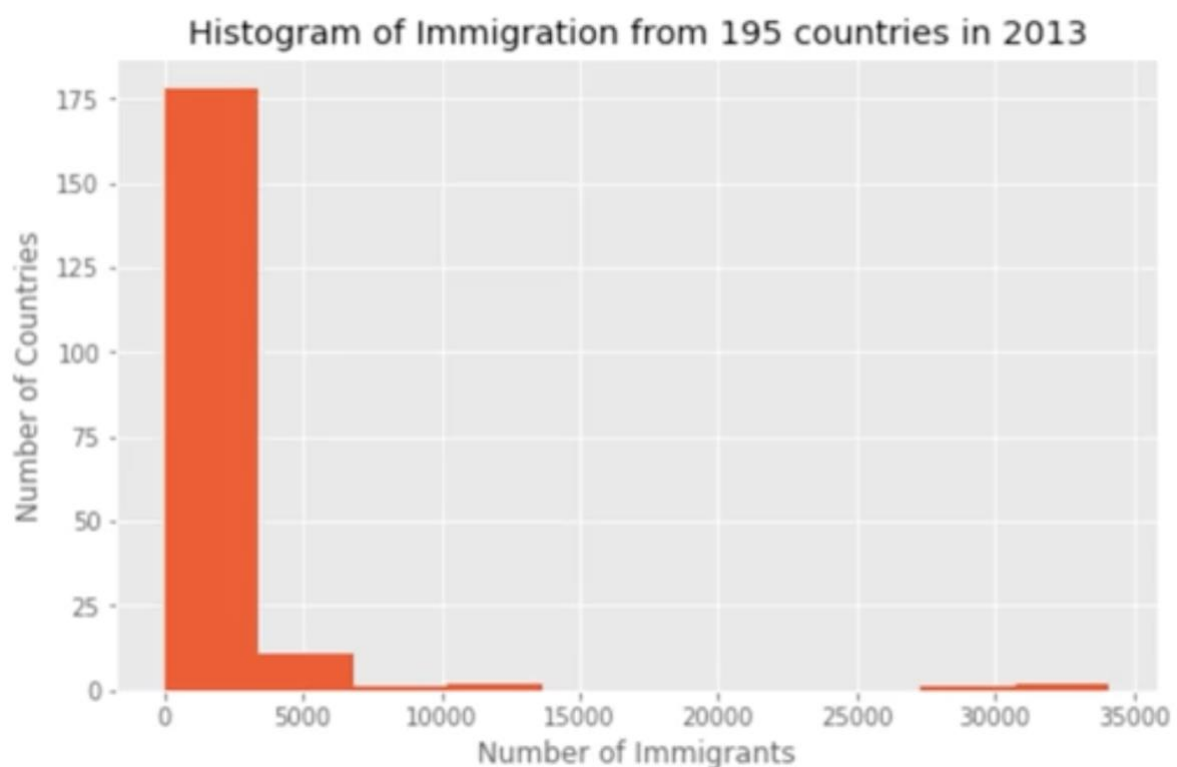
plt.title('Histogram of Immigration to Canada in 2013')
plt.xlabel('Number of Immigrants')
plt.ylabel('Number of Countries')

plt.show()
```

This produces a more precise histogram where:

- Bin edges
- Tick marks

are clearly aligned on the horizontal axis.



10.8 Summary

A histogram is used to represent the frequency distribution of a numeric data set.

Key points:

- Histograms divide data into bins
- The vertical axis represents frequency
- Matplotlib can generate histograms using:

```
kind='hist'
```

- NumPy's histogram function can be used to create properly aligned bins for better readability

```
count, bin_edges = np.histogram(data)
```

11 Bar Charts

11.1 Introduction to Bar Charts

A bar chart, also known as a bar graph, is a popular visualization tool. Unlike a histogram, a bar chart is a type of plot where the length of each bar is proportional to the value of the item that it represents.

Bar charts are commonly used to compare the values of a variable at a given point in time.

11.1.1 Example

Suppose we want to visualize, in a discrete fashion, how immigration from Iceland to Canada looked from 1980 to 2013.

One way to do this is by building a bar chart where the height of each bar represents the total immigration from Iceland to Canada in a particular year.

11.2 Creating a Bar Chart Using Matplotlib

From the immigration dataset for Canada, a dataframe named `df_canada` is created.

11.2.1 Structure of `df_canada`

- Country names are used as the index of each row
- A column named `Total` represents the cumulative sum of annual immigration from each country from 1980 to 2013

To visualize immigration from Iceland to Canada, a new dataframe named `df_iceland` is created using the `years` variable.

The dataframe includes:

- Annual immigration data for Iceland
- Excludes the Total column

11.3 Generating a Bar Chart

The `plot()` function is used on `df_iceland`, and the parameter `kind='bar'` is specified to generate a bar chart.

After creating the chart:

- A title is added
- Both axes are labeled
- The `show()` function is used to display the figure

11.3.1 Example Code

```
import matplotlib.pyplot as plt
```

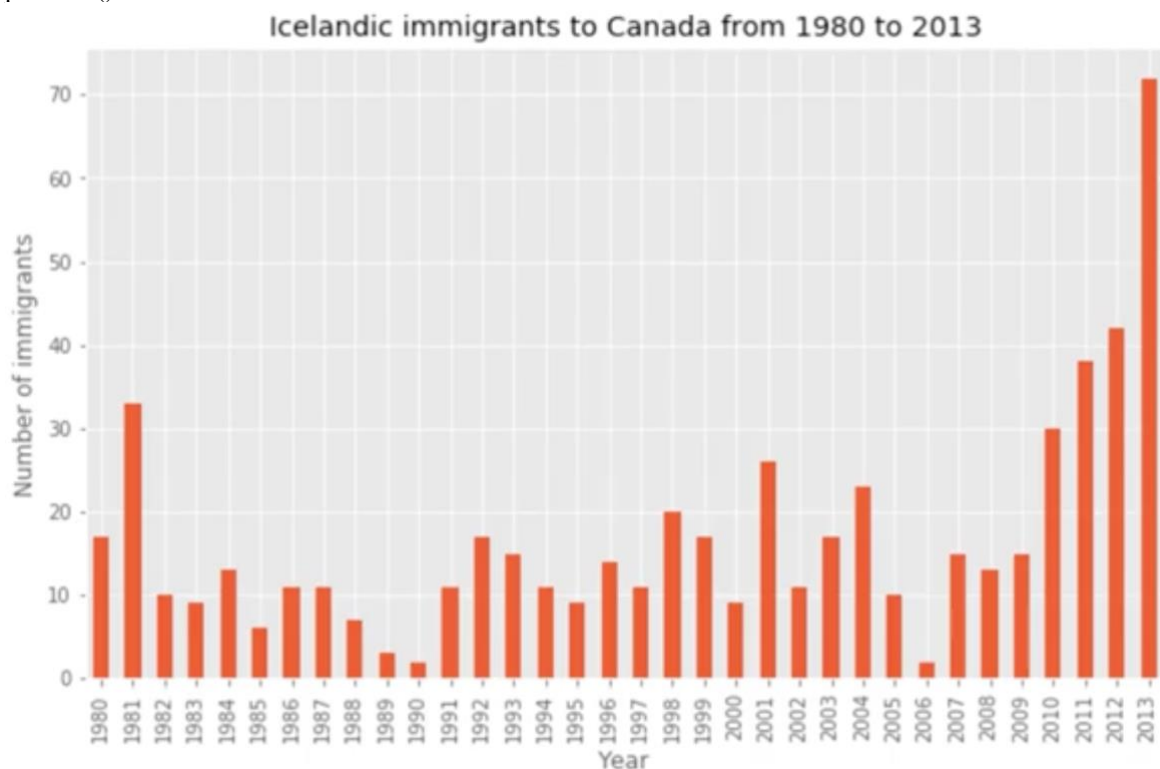
```
df_iceland.plot(kind='bar')
```

```
plt.title('Icelandic Immigration to Canada from 1980 to 2013')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Number of Immigrants')
```

```
plt.show()
```



11.4 Observations from the Bar Chart

The bar chart shows immigration from Iceland to Canada between 1980 and 2013.

By examining the chart, we can observe that immigration to Canada from Iceland has increased since 2010.

11.5 Horizontal Bar Charts

A horizontal bar chart can also be created by assigning 'barh' (bar horizontal) to the kind parameter of the plot() function.

11.5.1 Example

```
df_iceland.plot(kind='barh')
```

11.6 Customizing Bar Colors

The color parameter can be used to change the color of the bars.

11.6.1 Highlighting Specific Years

Suppose we want to highlight the years with the highest and lowest number of Icelandic immigrants to Canada between 1980 and 1990.

A list of colors can be passed to the color parameter accordingly.

In this example:

- The bars for the years 1981 and 1990 are highlighted in red

11.6.2 Example

```
colors = ['blue', 'red', 'blue', 'blue', 'blue', 'blue', 'blue', 'blue', 'blue', 'red']
```

```
df_iceland.plot(kind='bar', color=colors)
```

11.7 Changing Bar Border Colors

The border color of each bar can be changed using the edgecolor parameter.

11.7.1 Example

```
df_iceland.plot(kind='bar', edgecolor='black')
```

11.8 Summary

A bar chart is a type of plot where the length of each bar is proportional to the value of the item it represents.

Using Matplotlib, bar charts can be created to visualize data such as immigration trends from Iceland to Canada. The charts can also be customized using:

- Horizontal bars
- Different bar colors
- Highlighted bars
- Custom border colors

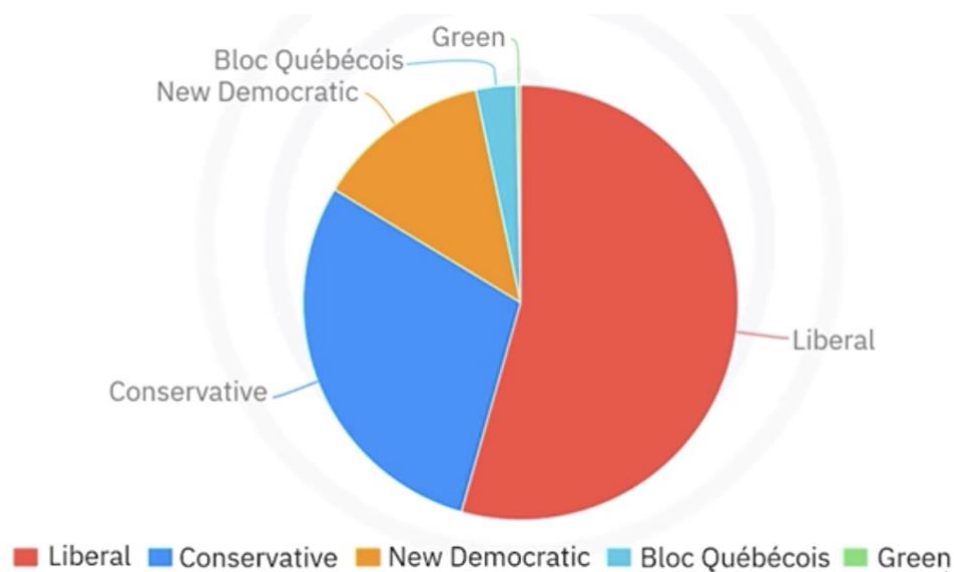
12 Pie Charts

12.1 What is a Pie Chart?

A pie chart is a **circular statistical graphic** divided into segments (slices) to illustrate **numerical proportions**.

12.1.1 Example

A pie chart of a Canadian federal election can be used to represent the **party-wise percentage of seats won in the House of Commons**. Each slice of the pie represents a party's share of the total seats.



12.2 Creating a Pie Chart Using Matplotlib

12.2.1 Dataset Example

To visualize the **continent-wise breakdown of immigration to Canada**, the dataset `df_canada` is used.

12.2.2 Step 1: Group the Data by Continent

Using the `Continent` column, group the data and calculate the total number of immigrants for each continent.

```
df_continents = df_canada.groupby('Continent').sum()
```

12.2.3 Resulting DataFrame

The resulting DataFrame (`df_continents`) contains:

- 6 rows, each representing a continent.
- 35 columns representing the years from **1980 to 2013**.
- A **Total** column containing the cumulative sum of immigration for each continent.

12.2.4 Step 2: Import Matplotlib

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

12.2.5 Step 3: Create the Pie Chart

Generate a pie chart using the Total column of df_continents.

```
df_continents['Total'].plot(
    kind='pie'
)

plt.title('Immigration to Canada by Continent')
plt.show()
```

12.2.6 Output

The resulting pie chart depicts each continent's proportion of immigration to Canada from **1980 to 2013**.

12.3 Exploding Pie Chart Slices

Matplotlib provides the **explode** property, which allows one or more slices to be offset from the center of the pie chart.

12.3.1 Purpose of Explode

- Highlights specific segments of the pie chart.
- Draws attention to important categories.
- Creates visual separation between slices.

12.3.2 How It Works

By assigning values to the `explode` parameter, you can control the degree of separation for each slice.

For example:

- Continents contributing less than **10%** of total immigration can be exploded outward.
- This makes smaller segments easier to identify and compare.

12.3.3 Example

```
explode = (0, 0, 0.1, 0.1, 0, 0)
```

```
df_continents['Total'].plot(
    kind='pie',
    explode=explode
)

plt.title('Immigration to Canada by Continent')
plt.show()
```

12.4 Limitations of Pie Charts

Pie charts are a popular visualization tool, but they also have critics.

Some data visualization experts argue that:

- Pie charts do not always display data accurately or consistently.
- Comparing proportions between slices can be difficult.
- Human perception is less effective at comparing angles and areas than comparing lengths.

12.4.1 Why Bar Charts Are Often Preferred

Bar charts generally perform better for:

- Accurate comparison of values.
- Clear communication of trends and differences.
- Consistent interpretation of data.

Because bar lengths are easier to compare than pie slices, bar charts are often recommended for analytical purposes.

12.5 Summary

A pie chart:

- Is a circular statistical graphic divided into segments.
- Illustrates numerical proportions.
- Can be created in Matplotlib using `kind='pie'`.
- Can highlight important segments using the `explode` parameter.
- Is useful for showing part-to-whole relationships.
- May be less effective than bar charts when an accurate comparison of values is required.

13 Box Plots

13.1 What is a Box Plot?

A **box plot** is a way of statistically representing the distribution of given data through **five primary dimensions**.

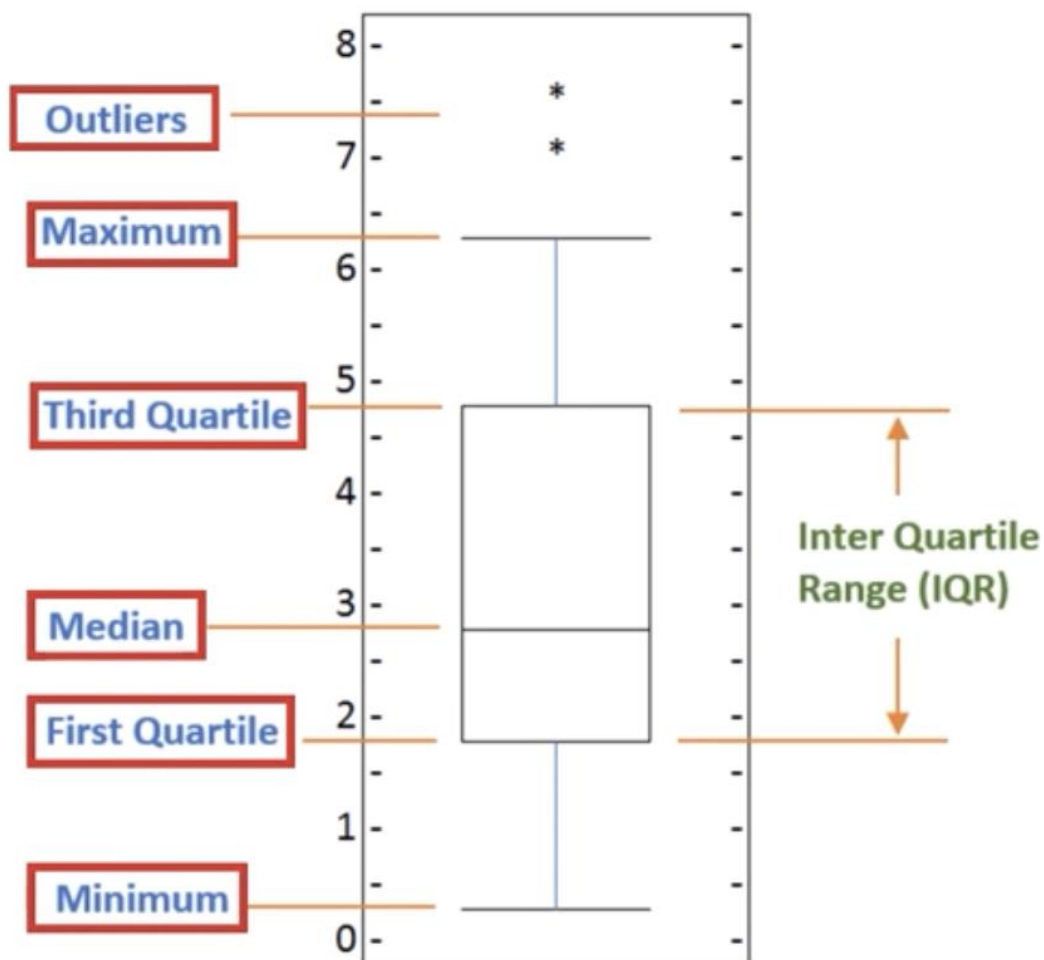
13.1.1 Five Main Dimensions of a Box Plot

- **Minimum**
 - Usually, the value is obtained by subtracting $1.5 \times \text{Interquartile Range (IQR)}$ from the first quartile.
- **First Quartile (Q1)**
 - The point 25% of the way through the sorted data.
 - In other words, a quarter of the data points are less than this value.
- **Median**

- The median of the sorted data.
- **Third Quartile (Q3)**
 - The point 75% of the way through the sorted data.
 - In other words, three-quarters of the data points are less than this value.
- **Maximum**
 - Usually, the value is obtained by adding $1.5 \times \text{IQR}$ to the third quartile.

13.1.2 Outliers

Box plots also display **outliers** as individual dots outside the upper and lower extremes of the plot.



13.2 Creating a Box Plot Using Matplotlib

To create a box plot for visualizing immigration from Japan to Canada:

13.2.1 Prepare the Data

- Process the dataframe `df_canada`.
- Set the country name as the index.
- Add a column representing the cumulative sum of annual immigration from each country from 1980 to 2013.

13.2.2 Import Required Libraries

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

13.2.3 Create a DataFrame for Japan

- Create a new dataframe containing data for Japan.
- Exclude the Total column using the years variable.
- Transpose the resulting dataframe into the correct format required for creating a box plot.
- Name the new dataframe df_japan.

13.2.4 Generate the Box Plot

Call the plot() function on df_japan and set kind='box'.

```
df_japan.plot(kind='box')
```

13.2.5 Complete the Figure

- Add an appropriate title.
- Label the vertical axis.
- Use the show() function to display the figure.

```
plt.title('Box Plot of Japanese Immigration to Canada (1980–2013)')
plt.ylabel('Number of Immigrants')
plt.show()
```

13.3 Interpretation of the Box Plot

The resulting box plot provides a good representation of the distribution of Japanese immigration to Canada from 1980 to 2013.

13.3.1 Observations

- There are **no outliers** in the data.
- The **median is closer to the top of the box**, indicating greater data concentration in the upper half of the distribution.

13.4 Key Takeaways

- A **box plot** is a statistical visualization used to represent data distribution.
- The five main dimensions of a box plot are:
 - Minimum
 - First Quartile (Q1)
 - Median
 - Third Quartile (Q3)
 - Maximum
- Outliers are displayed as individual points outside the whiskers.
- Box plots can be created in Matplotlib using:

```
plot(kind='box')
```

- Box plots are useful for understanding the spread, central tendency, and potential outliers within a dataset.

14 Scatter Plots

14.1 What is a Scatter Plot?

A scatter plot is a type of plot that displays values pertaining to typically two variables against each other.

Usually, a dependent variable is plotted against an independent variable to determine whether any correlation (relationship between variables) exists between the two variables.

14.1.1 Example: Income vs. Education

A scatter plot of income versus education can help identify relationships between these two variables.

By looking at the plotted data, one can conclude that an individual with more years of education is likely to earn a higher income than an individual with fewer years of education.

14.2 Creating a Scatter Plot with Matplotlib

Consider the DataFrame **df_canada**, where:

- Country names are set as the index.
- One column, **total**, represents the cumulative sum of annual immigration from each country from **1980 to 2013**.

Suppose we want to create a scatter plot showing the total annual immigration to Canada from **1980 to 2013**.

14.2.1 Preparing the Data

Before creating the scatter plot, a new DataFrame needs to be created that contains:

- Each year.
- The corresponding total number of immigrants from all countries worldwide.

This new DataFrame is named **df_total**.

14.2.2 Importing Matplotlib

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

14.2.3 Creating the Scatter Plot

The `plot()` function is used on the DataFrame `df_total`, with `kind='scatter'` specified to generate a scatter plot.

Unlike some other visualization types, scatter plots require additional parameters:

- $x \rightarrow$ Variable displayed on the horizontal axis.
- $y \rightarrow$ Variable displayed on the vertical axis.

In this example:

- year is passed as the x parameter.
- total is passed as the y parameter.

Example:

```
df_total.plot(
    kind='scatter',
    x='year',
    y='total',
    color='darkblue'
)

plt.title('Total Immigration to Canada from 1980-2013')
plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.show()
```

After adding an appropriate title and axis labels, the `show()` function is used to display the figure.

The resulting scatter plot shows total immigration to Canada from countries all over the world between **1980 and 2013**.

14.3 Interpreting the Scatter Plot

The scatter plot clearly depicts an overall rising trend of immigration over time.

This indicates that immigration to Canada generally increased throughout the period from **1980 to 2013**.

14.4 Customizing Scatter Plots

14.4.1 Using the Color Parameter

The `color` parameter can be used to specify the color of the markers.

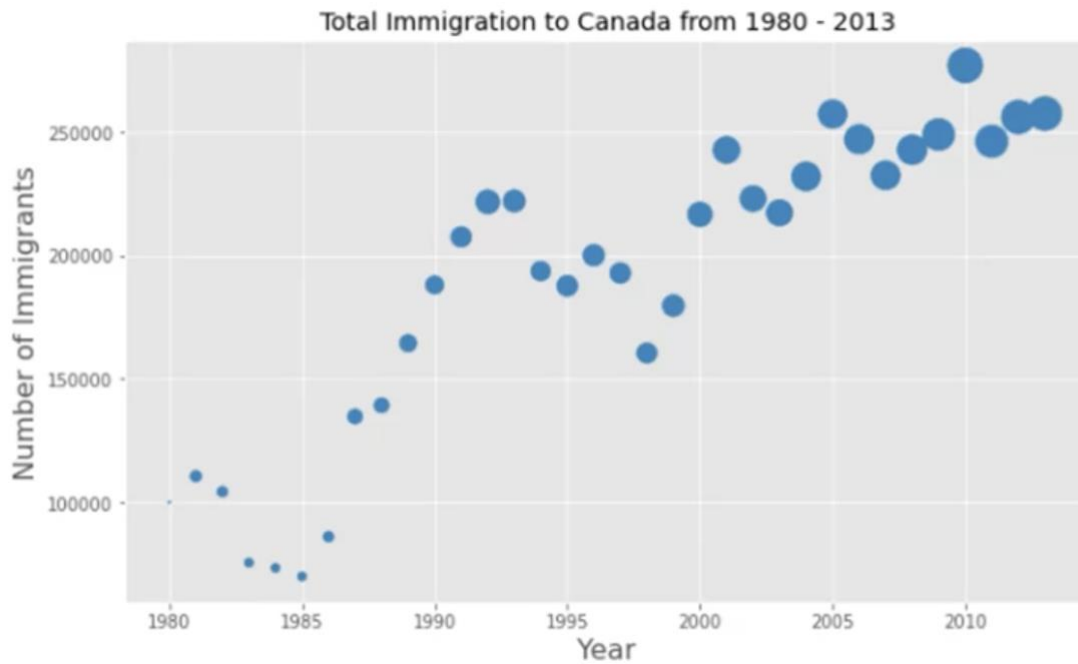
Example:

```
color='darkblue'
```

You may choose any color from the available color palettes.

For example:

```
color='red'
```



14.4.2 Using the Size Parameter (s)

The `s` parameter can be used to represent a third variable through marker size.

In the example discussed:

- The total immigration from the African continent is used as the third variable.
- Marker size increases according to the total number of immigrants from Africa.

As the years progress, the increasing marker sizes indicate that the number of immigrants from the African continent has increased over time.

Example:

```
df_total.plot(
    kind='scatter',
    x='year',
    y='total',
    s=africa_totals
)
```

14.5 Key Points

- A scatter plot displays values pertaining to typically two variables against each other.
- It is commonly used to identify correlations (relationships) between variables.
- Creating a scatter plot in Matplotlib requires:
 - Importing Matplotlib.
 - Specifying `kind='scatter'`.
 - Providing `x` and `y` parameters.
- The `color` parameter controls marker color.
- The `s` parameter allows marker size to represent a third variable.
- Scatter plots are useful for visualizing large datasets and identifying trends and relationships in data.

15 Understanding Treemaps and Pivot Charts

15.1 Treemaps

15.1.1 Introduction

Treemaps are a form of data visualization that displays hierarchical data using nested rectangles. Each branch of the hierarchy is represented by a rectangle, which is further divided into smaller rectangles representing sub-branches. Treemaps are particularly useful for visualizing large datasets where the hierarchical structure is important, providing an intuitive and space-efficient way to display data.



15.1.2 Applications of Treemaps

Treemaps are widely used across various domains for effectively communicating complex hierarchical data.

- Business Analytics: Visualizing the composition of sales by product categories and subcategories.
- Finance: Displaying the performance of stock portfolios, sectors, and industries.
- IT and Network Management: Representing file systems or network usage, showing the distribution of files and folders.
- Bioinformatics: Displaying hierarchical biological data, such as taxonomies or genomic structures.
- Website Analytics: Showing website traffic structure, where rectangles represent web pages and their size indicates the volume of visits.

15.1.3 Importance of Treemaps in Data Visualization

- Space Efficiency: Allows large datasets to be visualized within a limited area.
- Hierarchy Representation: Clearly shows both the structure and quantitative relationships within hierarchical data.
- Comparative Analysis: Makes it easy to compare the sizes of elements across different hierarchy levels.
- Immediate Insight: Rectangle size and color quickly highlight patterns, trends, and outliers.

15.1.4 Syntax for Generating a Treemap

15.1.4.1 Install Required Libraries

```
pip install plotly pandas
```

15.1.4.2 Import Libraries

```
import pandas as pd
```

```
import plotly.express as px
```

15.1.4.3 Load Data

```
# Replace with your actual dataset or data source
```

```
data = {  
    'Category': ['Category 1', 'Category 1', 'Category 2', 'Category 2', 'Category 3'],  
    'Subcategory': ['Subcategory 1A', 'Subcategory 1B', 'Subcategory 2A', 'Subcategory 2B', 'Subcategory 3A'],  
    'Value': [10, 20, 30, 40, 50]  
}
```

```
df = pd.DataFrame(data)
```

15.1.4.4 Create Treemap

```
fig = px.treemap(  
    df,  
    path=['Category', 'Subcategory'],  
    values='Value',  
    title='Treemap Example'  
)
```

15.1.4.5 Display Treemap

```
fig.show()
```

15.1.5 Practical Example: Visualizing Sales Data Using a Treemap

15.1.5.1 Sample Data

- Electronics
 - Laptops: 120,000
 - Smartphones: 80,000
 - Tablets: 30,000
- Furniture
 - Chairs: 50,000
 - Tables: 40,000
 - Sofas: 20,000
- Clothing
 - Men: 70,000
 - Women: 90,000
 - Kids: 40,000

15.1.5.2 Code to Generate Treemap

```
import plotly.express as px
```

```
import pandas as pd
```

```
# Sales data
```

```
data = {  
    'Category': ['Electronics', 'Electronics', 'Electronics',  
                'Furniture', 'Furniture', 'Furniture',  
                'Clothing', 'Clothing', 'Clothing'],  
    'Subcategory': ['Laptops', 'Smartphones', 'Tablets',  
                   'Chairs', 'Tables', 'Sofas',  
                   'Men', 'Women', 'Kids']  
}
```

```

        'Chairs', 'Tables', 'Sofas',
        'Men', 'Women', 'Kids'],
'Sales': [120000, 80000, 30000,
          50000, 40000, 20000,
          70000, 90000, 40000]
}

```

```
df = pd.DataFrame(data)
```

```

fig = px.treemap(
    df,
    path=['Category', 'Subcategory'],
    values='Sales',
    title='Sales Data Treemap'
)

```

```
fig.show()
```

15.1.5.3 Output

Sales Data Treemap



15.1.6 Conclusion

Treemaps are a powerful tool for visualizing hierarchical data. They provide an efficient and intuitive way to compare elements within a hierarchy while making optimal use of available space. Their applications span business analytics, finance, IT management, bioinformatics, and website analytics. By using Plotly Express, treemaps can be easily generated to gain insights and effectively communicate hierarchical relationships within data.

15.2 Pivot Charts

15.2.1 Introduction

Pivot charts are powerful tools used for data visualization and analysis. They enable users to dynamically summarize and explore large datasets, revealing insights and trends that may not be immediately apparent. Pivot charts are commonly used in business intelligence, finance, marketing, healthcare, and many other data-driven fields.

15.2.2 Applications of Pivot Charts

- Business Intelligence: Summarizes complex business data to support informed decision-making.
- Finance: Helps analysts visualize financial performance metrics and forecast trends.
- Marketing: Assists in analyzing campaign effectiveness, customer demographics, and sales performance.
- Operations: Supports monitoring of supply chain performance, inventory levels, and operational efficiency.
- Healthcare: Helps visualize patient data, treatment outcomes, and healthcare operations.

15.2.3 Importance of Pivot Charts in Data Visualization

- Data Summarization: Converts large datasets into manageable and understandable summaries.
- Dynamic Analysis: Enables filtering, sorting, and drilling down into specific data segments.
- Trend Identification: Helps uncover patterns and trends for strategic decision-making.
- Efficiency: Reduces the manual effort required to analyze large volumes of data.
- Presentation: Produces professional visualizations that can be shared with stakeholders.

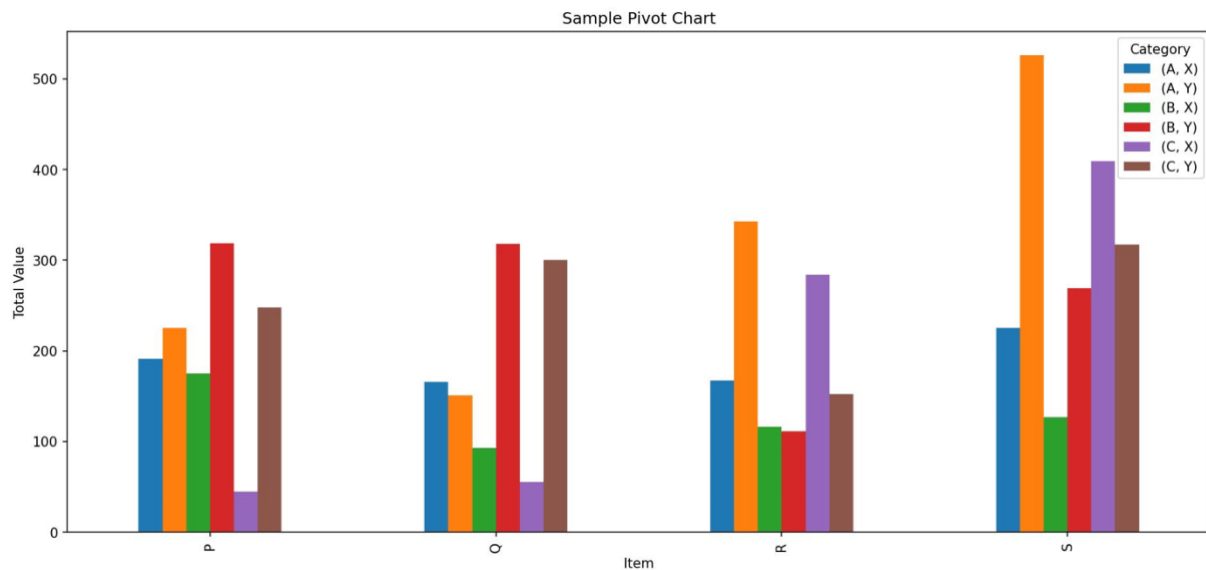
15.2.4 Sample Data Structure

Item	Category	Subcategory	Value
P	B	Y	62
S	C	Y	86
Q	A	X	98
P	A	Y	45
S	C	X	35
...	...	...	...
P	B	X	37
R	B	Y	26
S	A	Y	33
Q	A	Y	43
P	A	Y	15

In this dataset:

- The item acts as the row index.
- Categories and subcategories act as column indexes.
- Value acts as the aggregated measure.

A pivot chart can then be created to summarize and visualize this data.



15.2.5 Syntax for Generating Pivot Charts in Python

15.2.5.1 Install Required Libraries

`pip install pandas matplotlib`

15.2.5.2 Import Libraries

`import pandas as pd`
`import matplotlib.pyplot as plt`

15.2.5.3 Load Data

`data = pd.read_csv('your_dataset.csv')`

15.2.5.4 Create a Pivot Table

```
pivot_table = data.pivot_table(
    values='ValueColumn',
    index='RowIndexColumn',
    columns='ColumnIndexColumn',
    aggfunc='sum'
)
```

15.2.5.5 Generate Pivot Chart

`pivot_table.plot(kind='bar')`

`plt.title('Pivot Chart Title')`
`plt.xlabel('X-axis Label')`
`plt.ylabel('Y-axis Label')`

`plt.show()`

15.3 Practical Example: Visualizing Sales Data Using a Pivot Chart

15.3.1 Sample Dataset

S. No.	Date	Category	Subcategory	Sales
0	Q1	Peripherals	Accessories	2092
1	Q1	Software	Accessories	4695
2	Q1	Software	Components	3106

3	Q1	Desktops	Accessories	3527
4	Q1	Laptops	Software Suites	1182
...	...	...	...	...
2395	Q4	Desktops	Accessories	2557
2396	Q4	Software	Accessories	2626
2397	Q4	Desktops	Components	2427
2398	Q4	Software	Components	1768
2399	Q4	Peripherals	Components	1714

15.3.2 Code Used to Generate the Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

num_quarters = 4
num_categories = 4
num_subcategories = 3
num_samples = num_quarters * num_categories * num_subcategories * 50

np.random.seed(40)

dates = np.repeat(
    ['Q1', 'Q2', 'Q3', 'Q4'],
    50 * num_categories * num_subcategories
)

categories = np.tile(
    np.random.choice(
        ['Laptops', 'Desktops', 'Peripherals', 'Software'],
        size=num_quarters * 50
    ),
    num_categories * num_subcategories
)

subcategories = np.tile(
    np.random.choice(
        ['Accessories', 'Components', 'Software Suites'],
        size=num_quarters * 50
    ),
    num_categories * num_subcategories
)

sales_values = np.random.randint(1000, 5000, size=num_samples)

df = pd.DataFrame({
    'Date': dates,
    'Category': categories,
    'Subcategory': subcategories,
```

```
'Sales': sales_values
})
```

15.3.3 Create a Pivot Table

```
pivot_table = df.pivot_table(
    index='Date',
    columns=['Category', 'Subcategory'],
    values='Sales',
    aggfunc=np.sum
)
```

15.3.4 Generate Pivot Chart

```
pivot_table.plot(
    kind='bar',
    figsize=(14, 8)
)
```

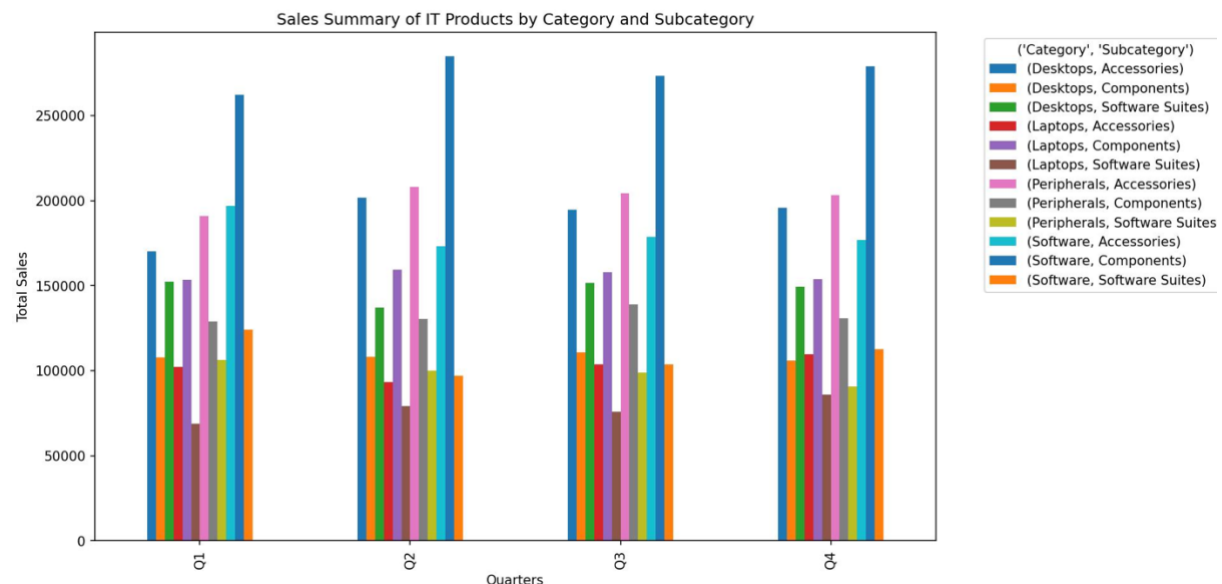
```
plt.title('Sales Summary of IT Products by Category and Subcategory')
plt.xlabel('Quarters')
plt.ylabel('Total Sales')
```

```
plt.grid(False)
```

```
plt.legend(
    title=('Category', 'Subcategory'),
    bbox_to_anchor=(1.05, 1),
    loc='upper left'
)
```

```
plt.tight_layout()
plt.show()
```

15.3.5 Output



15.3.6 Conclusion

Pivot charts are indispensable tools for data analysis because they provide a dynamic and intuitive way to summarize and visualize data. Using libraries such as Pandas and Matplotlib, pivot charts can be created with minimal effort, enabling users to uncover insights, identify trends, and support data-driven decision-making across various industries and applications.

16 Plotting Directly with Matplotlib

16.1 Introduction to Matplotlib

Matplotlib is a general-purpose, comprehensive plotting library that provides a flexible interface for creating a wide range of plots. Its **Pyplot module** offers a convenient way to create and customize plots quickly.

16.2 Importing Required Libraries

First, import the required libraries:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

16.2.1 Purpose of Each Library

- **Matplotlib Pyplot (plt)**: Used for creating and customizing visualizations.
- **NumPy (np)**: Commonly used as a data source for plotting and supports mathematical operations.
- **Pandas (pd)**: Used for handling tabular data.

16.3 Creating a Figure and Axes

Use the `subplots()` function to create:

- A **Figure** (Canvas window)
- One or more **Axes** (the area where plots are displayed)

```
fig, ax = plt.subplots()
```

The figure-axes pair provides greater control over the plotting area and overall figure customization.

16.4 Creating a Line Plot

16.4.1 Generating Sample Data

```
years = np.arange(1980, 2014)
immigrants = np.random.randint(2000, 10000, size=34)
```

16.4.2 Explanation

- `np.arange(1980, 2014)` generates a one-dimensional array from 1980 to 2013.
- `np.random.randint(2000, 10000, size=34)` generates 34 random integers between 2,000 and 10,000.

16.4.3 Plotting the Data

```
ax.plot(years, immigrants)
plt.show()
```

The `plot()` function creates a line plot using:

- X-axis values → years
- Y-axis values → immigrants

16.5 Creating a Scatter Plot

To display the same data as a scatter plot:

```
ax.scatter(years, immigrants)
plt.show()
```

The `scatter()` function plots individual data points instead of connecting them with lines.

16.6 Adding Titles and Axis Labels

16.6.1 Using Pyplot Functions

```
plt.title('Immigrants Between 1980 and 2013')
plt.xlabel('Years')
plt.ylabel('Total Immigrants')
```

16.6.2 Using Axes Methods

```
ax.set_title('Immigrants Between 1980 and 2013')
ax.set_xlabel('Years')
ax.set_ylabel('Total Immigrants')
```

These methods add meaningful titles and labels to improve readability.

16.7 Setting Axis Limits

The `xlim()` and `ylim()` functions allow you to control the visible range of the axes.

```
plt.xlim(start, end)
plt.ylim(start, end)
```

This helps focus attention on a specific portion of the data.

16.8 Adding Grid Lines

To improve readability:

```
ax.grid(True)
```

Grid lines make it easier to interpret values and trends.

16.9 Adding a Legend

To display a legend:

```
ax.legend(loc='best')
```

The `loc` parameter specifies where the legend appears on the figure.

Example:

```
ax.legend(loc='upper right')
```

16.10 Customizing Plots

Matplotlib provides several customization options.

16.10.1 Marker Styles

Markers represent individual data points.

Examples:

- 's' → Square marker
- 'o' → Circular marker (dot)

```
ax.plot(years, immigrants, marker='s')
```

For scatter plots:

```
ax.scatter(years, immigrants, s=50)
```

Where:

- The marker specifies the marker style.
- `s` specifies marker size in scatter plots.

16.10.2 Color Customization

Colors can be assigned to markers, lines, and plot elements.

```
ax.plot(years, immigrants, color='red')
```

16.10.3 Line Styles

Different line styles can be applied:

- Solid line
- Dashed line
- Dotted line

```
ax.plot(years, immigrants, linestyle='--')
```

16.11 Creating a Bar Plot

Bar plots are useful for comparing values across categories.

```
ax.bar(years, immigrants)
```

16.11.1 Explanation

- X-axis → Years
- Y-axis → Number of immigrants

The height of each bar represents the number of immigrants for that year.

16.12 Creating a Histogram

Histograms show the distribution of data.

```
ax.hist(
    immigrants,
    bins=20,
    color='blue',
    edgecolor='black'
)
```

16.12.1 Parameters

- bins=20 → Divides the data into 20 intervals.
- color → Sets the fill color.
- edgecolor → Sets the border color.

16.13 Creating a Pie Chart

Pie charts display proportions of a whole.

Example: Plotting immigrant data for five years (1980–1984).

```
ax.pie(
    values,
    labels=years,
    colors=colors,
    autopct='%1.1f%%'
)
```

16.13.1 Parameters

- labels → Labels for pie slices.
- colors → Colors assigned to each slice.
- autopct='%1.1f%%' → Displays percentages with one decimal place.

16.14 Creating Multiple Plots on the Same Figure

16.14.1 Using subplots()

Specify the number of rows and columns:

```
fig, axs = plt.subplots(
    1,
    2,
    sharey=True
)
```

)

16.14.2 Explanation

- 1 row
- 2 columns
- `sharey=True` allows both plots to use the same Y-axis.

16.14.3 Plotting on Individual Axes

```
axs[0].plot(years, immigrants)
```

```
axs[1].scatter(years, immigrants)
```

This creates:

- A line plot on the first axis.
- A scatter plot on the second axis.

16.15 Creating Multiple Subplots Using `add_subplot()`

Another approach is using `add_subplot()`.

16.15.1 Syntax

```
ax = figure.add_subplot(rows, columns, index)
```

16.15.2 Example

```
ax1 = figure.add_subplot(2, 2, 1)
```

16.15.3 Explanation

- 2 rows
- 2 columns
- 1 indicates the first subplot position

Create plots on the axis:

```
ax1.plot(years, immigrants)
```

Similarly, additional axes can be created:

```
ax2 = figure.add_subplot(2, 2, 2)
```

```
ax3 = figure.add_subplot(2, 2, 3)
```

```
ax4 = figure.add_subplot(2, 2, 4)
```

Different plots can then be displayed on all four axes within the same figure.

16.16 Data Storytelling vs Data Visualization

16.16.1 Data Storytelling

Data storytelling is the art of creating a narrative around data.

Characteristics:

- Builds a compelling story.
- Provides context and meaning.
- Helps communicate insights effectively.
- Focuses on audience engagement and decision-making.

16.16.2 Data Visualization

Data visualization is the process of creating visual representations of data.

Characteristics:

- Uses charts and graphs to communicate information.
- Helps identify patterns, trends, and relationships.
- Makes data easier to understand and explore.
- Brings data to life through visual elements.

16.16.3 Relationship Between the Two

Data visualization is an important component of data storytelling.

- **Data Visualization** focuses on presenting information visually.
- **Data Storytelling** combines visuals, context, and narrative to communicate a meaningful message.

16.17 Key Takeaways

- Matplotlib is a versatile plotting library used to create a wide variety of visualizations.
- The Pyplot module provides a convenient interface for creating and customizing plots.
- Common plot types include:
 - Line plots
 - Scatter plots
 - Bar plots
 - Histograms
 - Pie charts
- Plot customization options include:
 - Titles
 - Axis labels
 - Legends
 - Grid lines
 - Colors
 - Markers
 - Line styles
- Multiple plots can be displayed using:
 - `subplots()`
 - `add_subplot()`
- Data storytelling involves creating a narrative around data.
- Data visualization focuses on creating visual representations to understand patterns, trends, and relationships within data.

17 Summary: Basic and Specialized Visualization Tools

17.1 Key Concepts Covered

At this point, we've understood the following concepts related to basic and specialized data visualization tools.

17.1.1 Pie Charts

- A pie chart is a circular statistical graphic divided into segments to illustrate numerical proportions.
- The process of creating a pie chart involves importing Matplotlib and using it to represent data visually.

17.1.2 Box Plots

- A box plot is a statistical visualization used to represent the distribution of a dataset through five key dimensions:
 - Minimum
 - First Quartile (Q1)
 - Median
 - Third Quartile (Q3)
 - Maximum
- Box plots can be created using Matplotlib.

17.1.3 Scatter Plots

- A scatter plot displays values for two variables against each other, helping to identify relationships, patterns, or trends within the data.
- The process of creating a scatter plot involves importing Matplotlib and using it to visualize large datasets.

17.1.4 Matplotlib

- Matplotlib is a versatile plotting library that provides a flexible interface for creating a wide variety of visualizations.
- The Pyplot module of Matplotlib offers a convenient way to create and customize plots quickly and efficiently.

17.1.5 Data Storytelling

- Data storytelling is the art of creating a narrative around data to communicate insights effectively.
- Data visualization plays an important role in data storytelling by transforming data into engaging and meaningful visual representations.

Module 3: Advanced Visualizations and Geospatial Data

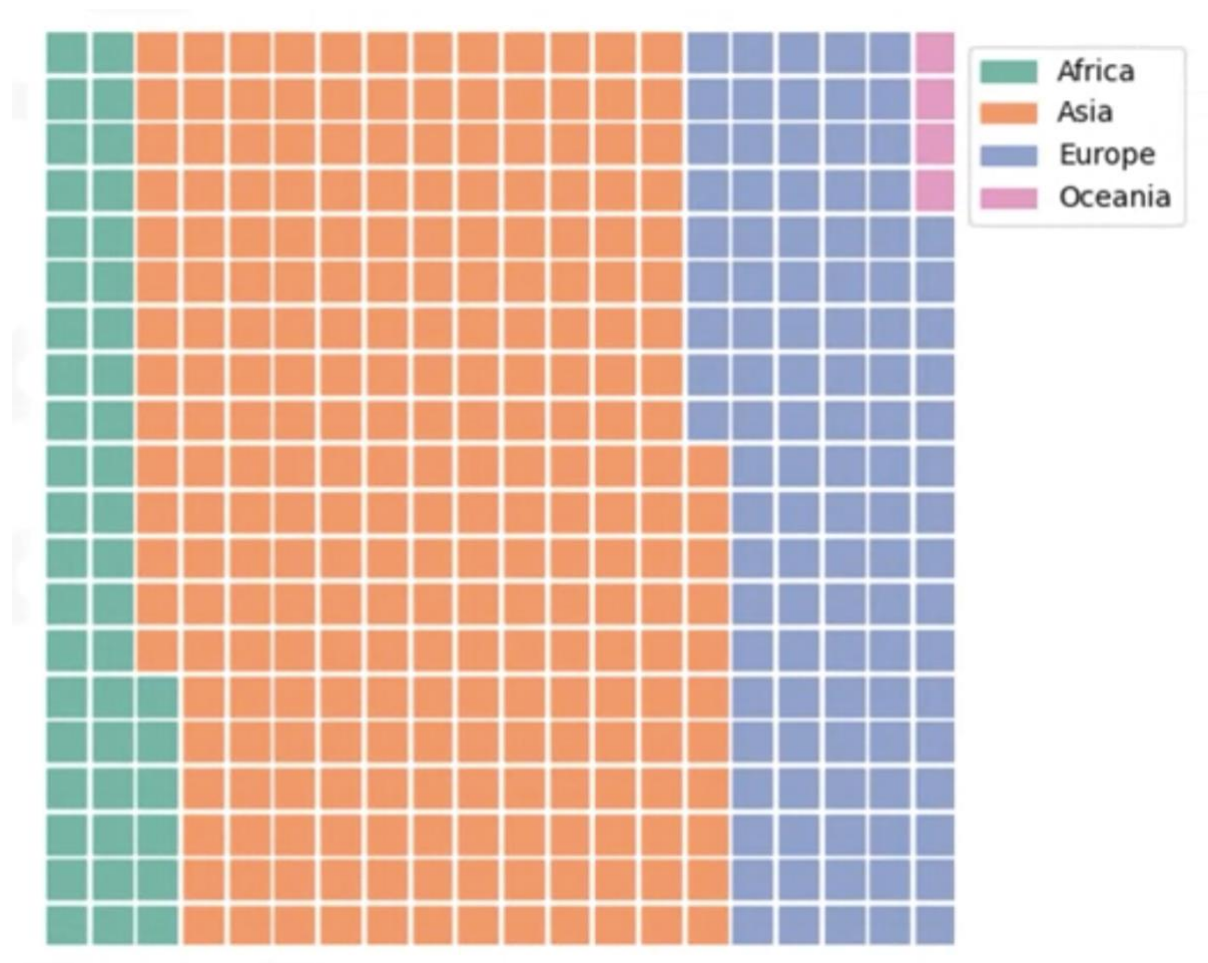
18 Waffle Charts and Word Cloud

18.1 Waffle Charts

18.1.1 Introduction

Waffle charts are a visualization technique that represents categorical data in the form of square tiles or cells.

- They resemble a grid of equal-sized squares.
- Each square represents a specific value or category.
- The size or color of the squares indicates the magnitude or proportion of each category.
- Waffle charts effectively show the proportion or percentage of different categories within an overall composition.
- The grid-like structure makes the data easy to understand and interpret, even for non-technical audiences.



18.1.2 Use Cases of Waffle Charts

Waffle charts can be used in the following areas:

- Market share analysis
- Demographic representation
- Project progress tracking
- Budget allocation
- Survey responses
- Election results
- Product sales analysis

18.1.3 Applications of Waffle Charts

18.1.3.1 Market Share Analysis

Visualize market share data by showing the proportion of companies or products within a specific market.

18.1.3.2 Demographic Representation

Display demographic data, such as age groups or ethnicities, within a population.

18.1.3.3 Project Progress Tracking

Represent the completion status of tasks or milestones, providing a visual overview of progress.

18.1.3.4 Budget Allocation

Demonstrate the allocation of budgetary resources across categories or departments within an organization.

18.1.3.5 Survey Responses

Summarize survey responses by displaying the distribution of answers to multiple-choice questions.

18.1.3.6 Election Results

Provide a clear visualization of voting outcomes by representing the distribution of votes for candidates or parties.

18.1.3.7 Product Sales Analysis

Illustrate product sales by representing the distribution of sales across categories or regions.

18.1.4 Creating Waffle Charts in Python

The **PyWaffle** library can be used to create visually appealing waffle charts that effectively communicate categorical data.

Key components include:

- Import the Waffle class from the PyWaffle library.
- Provide the `values` parameter with the data (for example, four continents and the total immigrants from the Canada Immigration dataset).
- Use the `rows` and `columns` parameters to determine the size of the chart grid.
- Customize the chart by adding a title and positioning the legend.

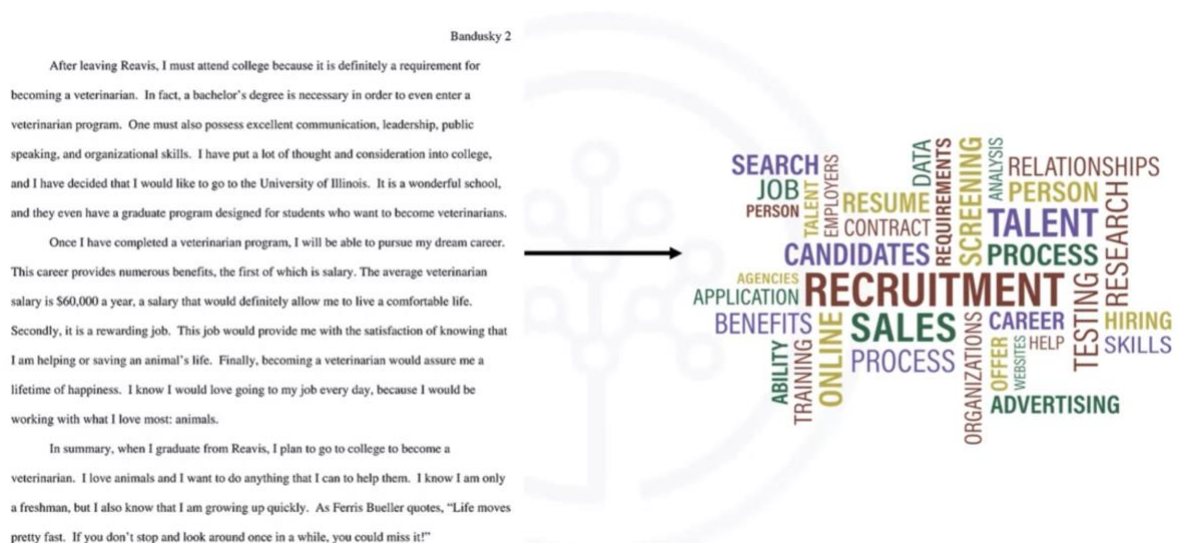
18.2 Word Cloud

18.2.1 Introduction

A word cloud, also known as a **tag cloud** or **text cloud**, is a popular data visualization method used to visually present textual data in an engaging and informative manner.

It provides a concise summary of textual content by offering a visual overview of the most commonly used words within a given text or collection of documents.

A word cloud is a depiction of the importance of different words in a body of text.



18.2.2 How Word Cloud Works

A word cloud works in a simple way:

- Words that appear more frequently in the source text are displayed larger and bolder.
- Words that appear less frequently are displayed in smaller.
- The size of each word reflects its frequency and importance within the text.

For example, in a recruitment-related document, words such as:

- Recruitment
- Talent
- Candidates

would appear larger because they occur more frequently in the text.

18.2.3 Use Cases of Word Cloud

Word clouds can be employed in the following areas:

- Social media analysis
- Customer feedback analysis
- Content analysis
- Market research
- Resume analysis
- Job description analysis

18.2.4 Applications of Word Cloud

18.2.4.1 Social Media Analysis

Extract and visualize popular topics or sentiments from social media conversations or hashtags.

18.2.4.2 Customer Feedback Analysis

Summarize customer reviews or feedback to identify recurring themes or sentiments.

18.2.4.3 Content Analysis

Analyze textual content such as articles, blogs, or research papers to uncover prevalent keywords or themes.

18.2.4.4 Market Research

Analyze survey responses, interviews, or focus group transcripts to extract key insights.

18.2.4.5 Resume and Job Description Analysis

Highlight important skills or keywords in resumes or job descriptions to assess their relevance.

18.3 Key Takeaways

- Waffle charts are a visualization technique that represents categorical data using square tiles or cells.
- Waffle charts effectively display proportions and percentages of categories within a dataset.
- Waffle charts are commonly used in market share analysis, demographics, project tracking, budget allocation, surveys, elections, and sales analysis.
- The PyWaffle library can be used in Python to create waffle charts.

- Word cloud is a popular data visualization technique used to visually represent textual data.
- The size of words in a word cloud depends on how frequently they appear in the source text.
- Word clouds are widely used in social media analysis, customer feedback analysis, content analysis, market research, and resume or job description analysis.

19 Seaborn and Regression Plots

19.1 Introduction to Seaborn

Seaborn is another data visualization library, but it is built on top of Matplotlib. It provides a higher-level interface for creating visually appealing and informative statistical graphics.

Some key features of Seaborn include:

- Offers a range of built-in themes and color palettes that improve the visual appeal of plots with minimal effort.
- Makes creating plots very efficient. With Seaborn, you can often generate plots with code that is five times less than with Matplotlib.
- Integrates well with statistical libraries such as NumPy and SciPy, allowing statistical analysis and visualizations to be easily combined.
- Provides specialized plot types such as:
 - Regression plots
 - Distribution plots
 - Categorical plots

While Pandas and Matplotlib are powerful tools for data manipulation and basic visualization, Seaborn complements them by providing a higher-level interface for creating statistical graphics. It works especially well when dealing with more complex visualizations and statistical analyses.

19.2 Regression Plots Using Seaborn

Consider a DataFrame called `df_total`, representing the total immigration to Canada from 1980 to 2013.

The DataFrame contains:

- Year column
- Total immigration column

The objective is to create:

- A scatter plot
- A regression line to highlight trends in the data

With Seaborn, this can be accomplished using a single line of code.

19.2.1 Import Seaborn

import seaborn as sns

19.2.2 Create a Regression Plot

```
sns.regplot(data=df_total, x='Year', y='Total')
```

The output of this single line of code includes:

- A scatter plot
- A regression line
- A 95% confidence interval (an estimated range that likely contains the true regression line)

19.3 Customizing a Regression Plot

The `regplot()` function accepts additional parameters for customization.

19.3.1 Change the Color

```
sns.regplot(  
    data=df_total,  
    x='Year',  
    y='Total',  
    color='green'  
)
```

19.3.2 Change the Marker Shape

The marker shape can be changed using the `marker` parameter.

For example, to use a plus (+) marker instead of the default circular marker:

```
sns.regplot(  
    data=df_total,  
    x='Year',  
    y='Total',  
    color='green',  
    marker='+'  
)
```

19.4 Plotting Categorical Data

The Canada immigration dataset contains several categorical features, such as:

- Country
- Region
- Continent

These categorical variables can also be visualized using Seaborn.

19.4.1 Count Plot

To represent the count of records for each continent, Seaborn provides the `countplot()` function.


```
sns.countplot(
    x='Continent',
    data=df_Canada
)
```

Parameters:

- x specifies the categorical variable to be plotted on the x-axis.
- data specifies the dataset to be used.

From the resulting plot, it is evident that most of the observations in the dataset are from Africa.

19.5 Bar Plot for Categorical Data

A bar plot can also be created using a slice of the df_Canada dataset.

The plot displays:

- Continent on one axis.
- Total immigration values on the other axis.

Seaborn automatically:

- Group the data by the categorical variable (Continent).
- Plots the aggregated values of Total.
- Displays confidence intervals for the aggregated values.

19.6 Key Takeaways

- Seaborn is a data visualization library based on Matplotlib.
- It was built primarily to provide a high-level interface for drawing statistical graphics.
- Seaborn allows scatter plots and regression lines to be created with a single line of code.
- The regplot() function provides:
 - Scatter plots
 - Regression lines
 - 95% confidence intervals
- The regplot() function also supports customization through parameters such as:
 - color
 - marker
- Seaborn provides specialized plots for statistical analysis, including:
 - Regression plots
 - Distribution plots
 - Categorical plots
- Functions such as countplot() and bar plots make it easy to visualize categorical data and aggregated values.

20 Introduction to Folium

20.1 What is Folium?

Folium is a powerful data visualization library in Python that was built primarily to help people visualize geospatial data (data related to geographic locations).

With Folium, you can:

- Create a map of any location in the world using latitude and longitude values.
- Create maps and superimpose markers and clusters (groups of nearby markers displayed together) on top of the map for interesting visualizations.
- Create maps with different styles, such as street-level maps, Stamen maps, and several others.

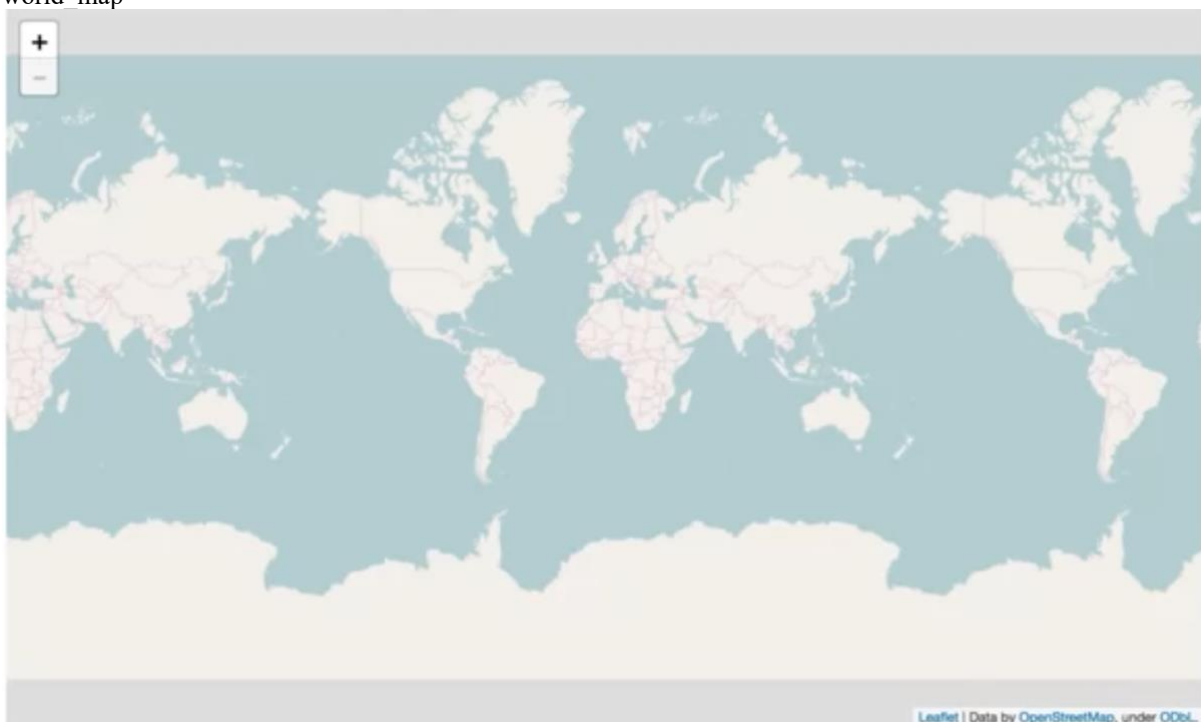
20.2 Creating a Basic Map with Folium

Creating a world map with Folium is straightforward.

First, import the Folium library and then call the Map() function.

```
import folium
```

```
world_map = folium.Map()  
world_map
```



20.3 Interactive Nature of Folium Maps

One of the most interesting features of maps created by Folium is that they are interactive.

After the map is rendered, you can:

- Zoom in.
- Zoom out.
- Navigate around the map.

This interactivity makes it easier to explore geographic data.

20.4 Default Map Style

The default map style in Folium is **OpenStreetMap**.

Features of the OpenStreetMap style:

- Shows a street-level view when zoomed in.
- Shows the borders of countries when zoomed out completely.

20.5 Creating a World Map Centered Around Canada

To create a map centered around Canada, pass Canada's latitude and longitude values using the `location` parameter.

You can also set the initial zoom level using the `zoom_start` parameter.

```
folium.Map(
    location=[56.130, -106.35],
    zoom_start=4
)
```

20.5.1 The `zoom_start` Parameter

The `zoom_start` parameter specifies the initial zoom level of the map.

It is called the initial zoom level because the user can easily change the zoom level after the map is rendered by zooming in or out.

You can experiment with different values to determine the most appropriate initial zoom level for your visualization.

For the map of Canada, the zoom level is set to:

```
zoom_start=4
```

The result is a world map centered around Canada.

20.6 Different Map Styles in Folium

Another powerful feature of Folium is the ability to create maps with different styles using the `tiles` parameter.

20.6.1 Stamen Toner Map

```
folium.Map(
```

```
location=[56.130, -106.35],
zoom_start=4,
tiles='Stamen Toner'
)
```

The **Stamen Toner** style is excellent for:

- Visualizing river meanders (curves and bends in rivers).
- Exploring coastal zones.

20.6.2 Stamen Terrain Map

```
folium.Map(
    location=[56.130, -106.35],
    zoom_start=4,
    tiles='Stamen Terrain'
)
```

The **Stamen Terrain** style is excellent for:

- Visualizing hill shading.
- Displaying natural vegetation colors.
- Exploring geographical terrain features.

20.7 Key Takeaways

- Folium is a Python data visualization library used for visualizing geospatial data.
- It allows you to create maps of any location using latitude and longitude coordinates.
- Folium maps are interactive and support zooming and navigation.
- The default map style is OpenStreetMap.
- You can center maps around specific locations using the `location` parameter.
- The `zoom_start` parameter controls the initial zoom level.
- Different map styles can be created using the `tiles` parameter, including:
 - OpenStreetMap
 - Stamen Toner
 - Stamen Terrain
- Folium also supports adding markers and clusters to create rich geographic visualizations.

21 Maps with Markers

21.1 Creating a Basic Map with Folium

With Folium, you can easily add markers to a map. To begin, create a world map centered around Canada.

21.1.1 Steps

- Import the Folium library.
- Create a map object.
- Specify the center point of the map using latitude and longitude coordinates.
- Set the initial zoom level using the `zoom_start` parameter.

In this example:

```
import folium
```

```
canada_map = folium.Map(  
    location=[56.1304, -106.3468],  
    zoom_start=4  
)
```

```
canada_map
```

21.1.2 Important Parameters

- **location:** Specifies the latitude and longitude coordinates of the center point of the map.
- **zoom_start:** Sets the initial zoom level of the map.

`zoom_start = 4` provides a zoomed-out view of Canada.

21.2 Understanding Markers

Markers play a vital role in enhancing interactivity and adding context to maps. They represent specific locations or points of interest and provide additional information when clicked.

Markers are like signposts that guide users through the map by highlighting important elements.

21.3 Adding a Single Marker

Ontario is Canada's most populous province and contains approximately 40% of the country's population. Let's add a marker representing Ontario.

21.3.1 Creating a Marker

```
folium.Marker(  
    location=[51.2538, -85.3232],  
    popup='Ontario'  
)add_to(canada_map)
```

```
canada_map
```

21.3.2 Explanation

- **location:** Specifies the approximate coordinates of Ontario.
- **popup:** Displays the label "Ontario" when the marker is clicked.
- **add_to(canada_map):** Adds the marker to the map so it becomes part of the map's layers and is displayed when the map is rendered or saved.

21.4 Creating Markers Using Feature Groups

Markers can also be created using a `FeatureGroup`.

21.4.1 Step 1: Create a Feature Group

```
ontario = folium.FeatureGroup(name='Ontario')
```

A newly created feature group is empty. The next step is to create children (map elements) and add them to the feature group.

21.4.2 Step 2: Create a Child Element

```
Ontario.add_child(  
    folium.CircleMarker(  
        location=[51.2538, -85.3232],  
        radius=5,  
        color='red',  
        fill=True  
    )  
)
```

This creates a red circular marker located at the center of Ontario.

21.4.3 Step 3: Add the Feature Group to the Map

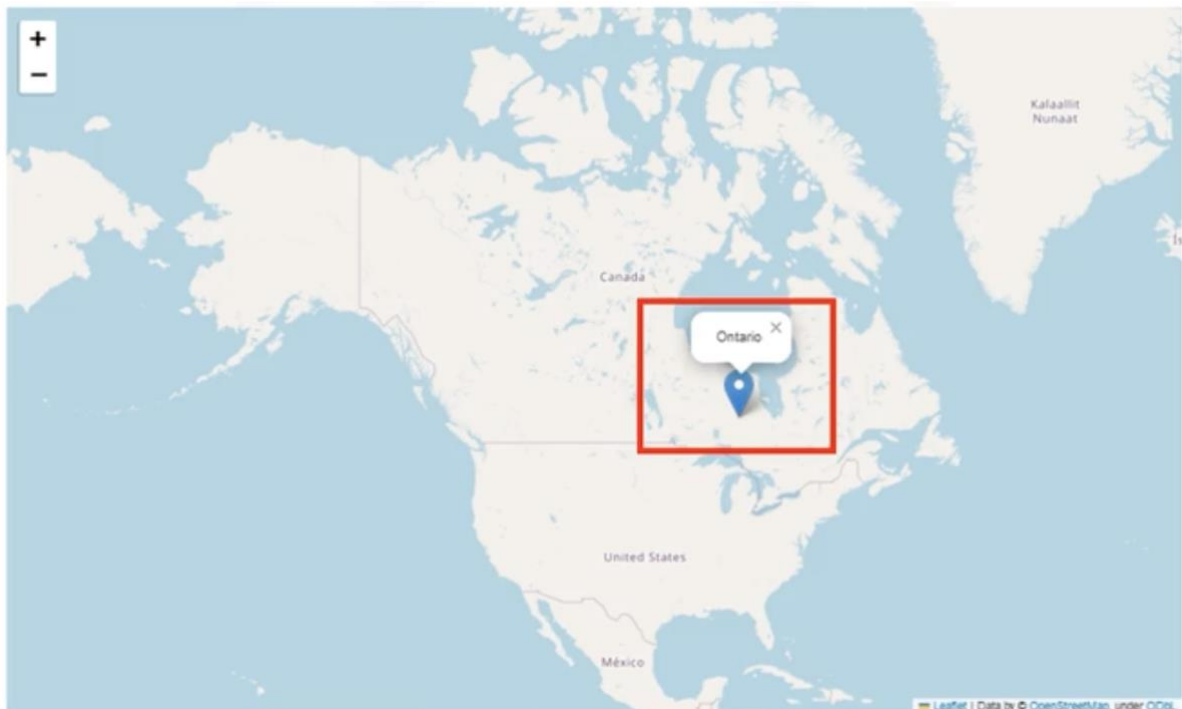
```
canada_map.add_child(ontario)
```

The result is a red circular marker superimposed on the map at the center of Ontario.

21.4.4 Adding a Label to the Marker

```
folium.Marker(  
    location=[51.2538, -85.3232],  
    popup='Ontario'  
)add_to(canada_map)
```

When the marker is clicked, it displays the label **Ontario**.



21.5 Displaying Multiple Markers

Displaying multiple markers becomes essential when working with larger maps.

21.5.1 Steps

- Create a list containing all locations.
- Loop through the list.
- Add a marker for each location.

Example:

```
locations = [  
    [51.2538, -85.3232],  
    [49.2827, -123.1207],  
    [45.4215, -75.6972]  
]
```

```
for location in locations:  
    folium.Marker(location).add_to(canada_map)
```

The map will then display multiple markers.

21.6 Using Marker Clusters

When a map contains many markers, it can become crowded and difficult to read. Folium provides `MarkerCluster` to solve this issue.

21.6.1 Creating a Marker Cluster

```
from folium.plugins import MarkerCluster  
  
marker_cluster = MarkerCluster().add_to(canada_map)
```

```
for location in locations:  
    folium.Marker(location).add_to(marker_cluster)
```

21.6.2 How Marker Clusters Work

The markers inside the `MarkerCluster` are intelligently grouped based on their proximity when the map is displayed.

21.6.3 Advantages of Marker Clustering

- Prevents overcrowding on the map.
- Improves visual presentation.
- Makes maps easier to interpret.
- Provides a clear representation when many markers are located close together.

21.7 Real-World Application

A practical application of marker clustering is the visualization of crime data in San Francisco. Thousands of markers can be superimposed on a map, and clustering helps make the visualization less congested and easier to understand.

21.8 Key Takeaways

- Folium makes it easy to add markers to maps.
- The location parameter specifies latitude and longitude coordinates.
- Markers enhance interactivity and provide context to maps.
- The folium.Marker() function is used to create markers.
- The popup parameter provides a label that appears when a marker is clicked.
- Markers can be created using FeatureGroup.
- Multiple markers can be generated using loops.
- MarkerCluster groups nearby markers and improves map readability when displaying a large number of markers.

22 Choropleth Maps

22.1 Introduction to Choropleth Maps

A **choropleth map** is a thematic map in which areas are shaded or patterned in proportion to the measurement of a statistical variable displayed on the map, such as:

- Population density
- Per capita income
- Infant mortality rate
- Immigration statistics

In a choropleth map:

- **Darker colors represent higher values.**
- **Lighter colors represent lower values.**

22.2 Examples of Choropleth Maps

22.2.1 Infant Mortality Rate Map

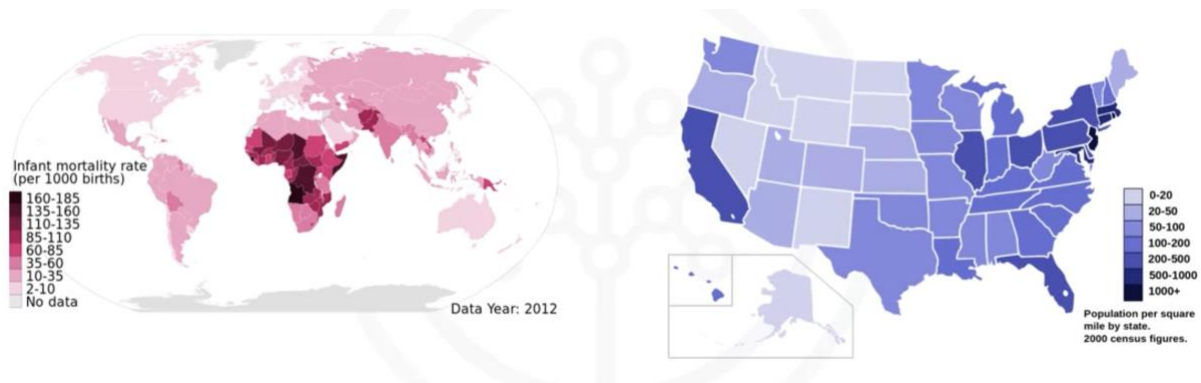
A choropleth world map can be used to display the **infant mortality rate per 1,000 births**.

- The darker the color, the higher the infant mortality rate.
- According to the map, many African countries have very high infant mortality rates.
- Some countries report rates higher than **160 deaths per 1,000 births**.

22.2.2 Population Density Map of the United States

Another example is a choropleth map showing the **population per square mile by state** in the United States.

- Darker colors indicate higher population density.
- States in the eastern part of the United States tend to be more populous than states in the western part.
- California is a notable exception, having a high population despite being located in the western part of the country.



22.3 Folium Library

Folium is a Python library used for creating interactive maps and visualizations.

It provides a simple and intuitive way to generate maps using data from various sources, including:

- GeoJSON files
- Pandas DataFrames
- NumPy arrays

22.4 GeoJSON Files

To create a choropleth map of a region of interest, Folium requires a **GeoJSON (Geographic JavaScript Object Notation)** file that contains the geospatial information of that region.

For example, to create a choropleth map of the world, the GeoJSON file should contain information about each country, including:

- Country name
- Country ID
- Geometry
- Shape
- Coordinates that define borders and boundaries

An example GeoJSON entry for a country such as **Brunei** would include all of these geographical details.

22.5 Example: Choropleth Map of Immigration to Canada

22.5.1 Dataset Overview

Consider a dataset containing immigration information to Canada.

Each row in the dataset represents a country and contains:

- Geographical metadata about the country
- Whether the country is developing or developed

- Annual immigration figures from that country to Canada from **1980 to 2013**

Assume that this dataset is stored in the DataFrame:

`df_canada`

22.6 Creating the Choropleth Map

22.6.1 Step 1: Create a World Map

First, create a world map using Folium.

This time, use the **Mapbox Bright Tileset**, which produces a world map displaying the name of every country.

22.6.2 Step 2: Convert the World Map into a Choropleth Map

To create the choropleth map:

- Define a variable that points to the GeoJSON file.
- Apply the `choropleth()` function to the world map.
- Use:
 - The `Country` and `Total` columns from `df_canada`.
 - Country names to look up geographical information from the GeoJSON file.

The resulting map shows the intensity of immigration to Canada from countries around the world.

Countries with larger immigration numbers are displayed with darker shades.

22.7 Key Points

- A **choropleth map** is a thematic map in which areas are shaded or patterned according to the value of a statistical variable.
- Darker colors generally represent higher values.
- Choropleth maps are useful for visualizing geographic distributions of data, such as:
 - Population density
 - Income levels
 - Mortality rates
 - Immigration statistics
- **Folium** is a Python library for creating interactive maps and visualizations.
- Folium requires a **GeoJSON file** containing geospatial information about the region being mapped.
- The **Mapbox Bright Tileset** displays the names of countries on the map, making it useful for world visualizations.

22.8 Summary

Choropleth maps provide an effective way to visualize statistical information geographically by using different shades or patterns to represent varying values. Using Folium together with

GeoJSON files, it is possible to create interactive choropleth maps that help identify geographic trends and patterns in data such as immigration, population, and health statistics.

23 Summary: Advanced Visualizations and Geospatial Data

23.1 Key Concepts Learned

At this point, we have learned the following concepts related to geospatial data visualization using Folium.

23.1.1 Folium

1. Folium is a data visualization library in Python that helps users visualize geospatial data.
2. With Folium, you can create maps of different styles, such as:
 1. Street-level maps
 2. Stamen maps
 3. Other custom map styles
3. Different map styles can be created using the `tiles` parameter.

23.1.2 Creating Maps

- The `location` parameter specifies the latitude and longitude coordinates of the center point of the map.
- Folium allows users to easily add markers to maps, making them more interactive and informative.

23.1.3 Markers in Folium

- Markers play a vital role in enhancing interactivity and adding context to maps.
- The `folium.Marker()` function is used to create markers on a map.
- The `location` parameter in `folium.Marker()` specifies the marker's latitude and longitude coordinates.
- The `popup` parameter provides a label or information box that appears when the marker is clicked.
- Multiple markers can be organized and created using a **Feature Group**.

23.1.4 Choropleth Maps

1. A choropleth map is a thematic map in which areas are shaded or patterned according to the value of a statistical variable.
2. When creating a choropleth map, Folium requires a **GeoJSON file** (a file containing geospatial data in JSON format) that defines the geographical boundaries of the regions being visualized.

23.1.5 Mapbox Bright Tileset

- The **Mapbox Bright Tileset** displays the name of every country when used on a map, making geographic regions easier to identify.

Module 4: Creating Dashboard with Plotly and Dash

24 Dashboarding Overview

24.1 Introduction

Dashboards provide a centralized and interactive way to visualize data and monitor business performance. With real-time visuals on a dashboard, understanding different moving parts of a business becomes easier. Based on the report type and data, suitable graphs and charts can be created in one central location.

Dashboards provide an easy way for stakeholders to understand:

- What is going right.
- What is going wrong.
- What improvements are necessary.

Having the big picture in one place helps businesses make informed decisions that can improve overall performance.

In general, the best dashboards answer critical business questions.

24.2 Example: Monitoring Domestic US Flight Performance

Suppose you are assigned the task of monitoring and reporting the performance of domestic US flights. The yearly review report includes:

- Top 10 airline carriers in the year 2019 in terms of the number of flights.
- Number of flights in 2019, split by month.
- Number of travelers from California state to other states, split by distance group.

24.3 Methods of Presenting Reports

24.3.1 Type 1: Tables and Documents

The report is presented through tables, and inferences from the tables are documented for reference.

Limitations:

- Time-consuming to create and analyze.
- Less visually appealing.

- Difficult to comprehend quickly.
- Not suitable for real-time data analysis.

24.3.2 Type 2: Dashboard Format

The same report is presented in the form of a dashboard.

Features of Dashboards:

- Hovering over charts provides detailed information about data points.
- Interactive charts allow users to explore data dynamically.
- Charts such as Sunburst Charts allow users to:
 - Click on different segments.
 - Drill down into different levels.
 - Obtain detailed information about each segment.

Dashboards make it easier to understand findings and interact with the data compared to static tables and reports.

24.4 Why Dashboards Are Important

A data scientist should have the ability to create and deliver a story around findings in a way that stakeholders can easily understand.

Dashboards help by:

- Simplifying complex business information.
- Providing real-time insights.
- Improving data comprehension.
- Supporting better decision-making.
- Making reports visually appealing and interactive.

Because of these advantages, dashboards are often the preferred way to present business data.

25 Web-Based Dashboarding Tools in Python

25.1 Dash

Dash is a Python framework for building web analytical applications.

Built on top of:

- Flask
- Plotly.js
- React.js

Features:

- Suitable for building data visualization applications.

- Supports highly customized user interfaces.
- Designed specifically for analytical web applications.

25.2 Panel

Panel works with visualizations created using:

- Bokeh
- Matplotlib
- HoloViews
- Many other Python plotting libraries

Features:

- Makes visualizations instantly viewable.
- Combines visualizations with interactive widgets.
- Works well inside Jupyter Notebooks for quick data exploration.
- Can also be deployed as a standalone dashboard application.
- Allows easy switching between notebook and deployed application environments.

25.3 Voila

Voila turns Jupyter Notebooks into standalone web applications.

Features:

1. Converts notebooks into dashboards without exposing code.
2. Compatible with layout tools such as:
 - Jupyter-flex
 - Voilà-vuetify templates

25.4 Streamlit

Streamlit can easily turn data scripts into shareable web applications.

25.4.1 Main Principles of Streamlit

- Embrace Python scripting.
- Treat widgets as variables.
- Reuse data and computation.

25.5 Other Dashboarding Tools

25.5.1 Bokeh

Bokeh is:

- A plotting library
- A widget library

- An application library

Features:

- Acts as a server for both plots and dashboards.
- Serves as the foundation on which the Panel is built.

25.5.2 Ipywidgets

Ipywidgets provides:

- A wide variety of Jupyter-compatible widgets.
- Interfaces supported by many Python libraries.

Limitation:

- Sharing as a dashboard requires a separate deployable server, such as Voila.

25.5.3 Matplotlib

Matplotlib is a comprehensive library for creating:

- Static visualizations
- Animated visualizations
- Interactive visualizations in Python

25.5.4 Bowtie

Bowtie allows users to build dashboards entirely in pure Python.

25.5.5 Flask

Flask is a Python-backed web server used to build arbitrary websites, including websites containing Python visualizations that function as Flask dashboards.

25.6 Key Takeaways

- Dashboards simplify the dynamic aspects of business.
- Interactive dashboards provide better insights than static reports and tables.
- Dashboards enable stakeholders to understand business performance and make informed decisions quickly.
- Python offers several tools for building dashboards, including:
 - Dash
 - Panel
 - Voila
 - Streamlit
 - Bokeh
 - Ipywidgets
 - Matplotlib

- Bowtie
 - Flask
- Various dashboarding tools are available, and the choice of tool depends on the application's requirements and the level of interactivity required.

26 Introduction to Plotly

26.1 What is Plotly?

Plotly is an interactive, open-source plotting library that supports more than 40 unique chart types. It is available in both Python and JavaScript.

Plotly Python is an extension of the Plotly JavaScript library and includes chart types such as:

- Statistical charts
- Financial charts
- Maps
- Scientific visualizations
- Three-dimensional (3D) data visualizations

The web-based visualizations created using Plotly Python can be:

- Displayed in Jupyter Notebook
- Saved as standalone HTML files
- Served as part of pure Python-built web applications using Dash.

26.2 Plotly Sub-Modules

The two important sub-modules of Plotly are:

- Plotly Graph Objects
- Plotly Express

26.2.1 Plotly Graph Objects

Plotly Graph Objects is the low-level interface to:

- Figures
- Traces
- Layouts

The `plotly.graph_objects` module provides an automatically generated hierarchy of classes called graph objects, which are used to represent figures with a top-level class:

```
plotly.graph_objects.Figure
```

26.2.2 Plotly Express

Plotly Express is a high-level wrapper for Plotly and is the recommended starting point for creating the most common figures provided by Plotly.

Features of Plotly Express:

- Simple syntax
- Easy to create visualizations
- Easy to modify charts
- Uses Graph Objects internally
- Automatically creates interactive visualizations

26.3 Creating a Line Chart Using Plotly Graph Objects

26.3.1 Import Required Libraries

```
import plotly.graph_objects as go
import plotly.express as px
import numpy as np
```

26.3.2 Generate Sample Data

```
np.random.seed(10)
```

The random seed is set for reproducibility (producing the same random values every time the code is executed).

Create an array containing 12 elements:

```
x = np.arange(12)
```

Generate random y-values:

```
y = np.random.randint(50, 500, size=12)
```

26.4 Creating the Figure

The `plotly.graph_objects` module contains a JSON object (data stored in a dictionary-like structure). Since the module was imported as `go`, `go` acts as the JSON object.

Create a figure by adding a trace (Scatter):

```
fig = go.Figure(
    data=go.Scatter(
        x=x,
        y=y
    )
)
```

26.5 Updating the Layout

The figure layout can be updated using the `update_layout()` method.

```
fig.update_layout(
    title='Simple Line Plot',
    xaxis_title='Month',
    yaxis_title='Sales'
)
```

Display the figure:

```
fig.show()
```

26.6 Creating the Same Chart Using Plotly Express

The entire line chart can be created using a single command.

```
fig = px.line(  
    x=x,  
    y=y,  
    title='Simple Line Plot',  
    labels=dict(  
        x='Month',  
        y='Sales'  
    )  
)  
fig.show()
```

26.6.1 Advantages of Plotly Express

- Requires less code.
- Creates visualizations quickly.
- Visualizations are automatically interactive.
- Easy to create and modify charts.

26.7 Airline Reporting Dataset

The Plotly library can be demonstrated using the Airline Reporting Dataset from the Data Asset Exchange.

26.7.1 Overview of the Dataset

The Reporting Carrier On-Time Performance Dataset contains information on approximately 200 million domestic US flights reported to the United States Bureau of Transportation Statistics.

The dataset contains information such as:

- Date
- Time
- Departure airport
- Arrival airport
- Length of flight delay (if applicable)
- Reason for the delay

26.8 Key Takeaways

- Plotly is an interactive, open-source plotting library that supports more than 40 chart types.
- Plotly Python is an extension of the Plotly JavaScript library.
- Plotly Graph Objects is the low-level interface to figures, traces, and layouts.
- Plotly Express is a high-level wrapper for Plotly.

- Plotly Express uses Graph Objects internally and provides a simpler way to create interactive visualizations.

27 Introduction to Dash

27.1 What is Dash?

Dash is an open-source user interface (UI) Python library for creating reactive, web-based applications. It is both enterprise-ready and a first-class member of Plotly's open-source tools.

Dash applications are:

- Web servers running Flask.
- Communicating JSON packets over HTTP requests.
- Rendering front-end components using React.js.

27.2 Why Use Dash?

It is easy to build graphical user interfaces (GUIs) using Dash because it abstracts (hides the underlying complexity of) all the technologies required to create web applications.

Dash has the following characteristics:

- Declarative (you describe what the UI should look like rather than how to build it step by step).
- Reactive (the UI automatically updates when data or inputs change).
- Can be rendered in a web browser.
- Can be deployed to servers.
- Provides a simple reactive decorator for binding code to the user interface.
- Inherently mobile and cross-platform ready.

27.3 Building an Application with Dash

Suppose you want to create an application to answer business questions.

The first step is to determine the application's layout:

- Decide which charts to use.
- Determine where to place them on the page.

This is known as the **layout part** of Dash.

The second step is to add **interactivity** to the application.

27.4 Components of Dash

Dash consists of two main components:

1. Core Components

2. HTML Components

27.4.1 Importing Core Components

Core components can be imported as `dcc` using the following statement:

```
from dash import dcc
```

27.4.2 Importing HTML Components

HTML components can be imported as `html` using the following statement:

```
from dash import html
```

27.5 Dash Core Components

The `dash_core_components` library describes higher-level interactive components generated with JavaScript, HTML, and CSS through the React.js library.

Examples of core components include:

- Slider
- Input area
- Check items
- Date picker

Many other components are also available in the Dash documentation.

27.6 Dash HTML Components

The `dash_html_components` library contains a component for every HTML tag.

You can compose your application's layout using Python structures together with the Dash HTML components library.

The `dash_html_components` library provides classes for all HTML tags, and the keyword arguments describe HTML attributes such as:

- `style`
- `className`
- `id`

No knowledge of HTML or CSS is required to use Dash, although having knowledge of these technologies can help in styling dashboards.

27.7 Key Takeaways

- Dash is an open-source user interface Python library for creating reactive, web-based applications.
- Dash makes it easy to build graphical user interfaces by abstracting the underlying web technologies.

- Dash applications use Flask on the backend and React.js on the frontend.
- Dash is declarative, reactive, and cross-platform ready.
- Dash has two major components:
 - Core Components (dcc)
 - HTML Components (html)
- The `dash_core_components` library provides higher-level interactive components.
- The `dash_html_components` library provides a component for every HTML tag.

28 Dash Basics: HTML and Core Components

28.1 Introduction

Dash is a Python framework used to build interactive web-based dashboards and data visualization applications. It integrates seamlessly with Plotly and provides HTML and core components that help in creating interactive user interfaces.

28.2 Import Required Packages

```
import pandas as pd
import plotly.express as px
import dash
from dash import dcc
from dash import html
```

28.2.1 Package Description

- **pandas (pd):** Used for data manipulation and analysis.
- **plotly.express (px):** Used to create interactive visualizations with minimal code.
- **dash:** Main framework for building web applications.
- **dash.dcc (Dash Core Components):** Provides interactive components such as graphs, dropdowns, sliders, and input boxes.
- **dash.html (HTML Components):** Provides HTML elements such as headings, paragraphs, divisions, buttons, etc.

28.3 Load the Dataset

```
# Read the airline data into pandas dataframe
airline_data = pd.read_csv("8.1_airline_data.csv")
```

The dataset is loaded into a Pandas DataFrame named `airline_data`.

28.4 Sample the Data

```
# Randomly sample 500 data points.
# random_state=42 ensures reproducible results.
data = airline_data.sample(n=500, random_state=42)
```

28.4.1 Explanation

- `n=500` selects 500 random rows from the dataset.
- `random_state=42` ensures that the same random sample is generated every time the code is executed.

28.5 Create a Pie Chart

```
fig = px.pie(
    data,
    values='Flights',
    names='DistanceGroup',
    title='Distance group proportion by flights'
)
```

28.5.1 Parameters

- **data:** DataFrame containing the data.
- **values='Flights':** Determines the size of each pie slice using the Flights column.
- **names='DistanceGroup':** Creates categories based on the DistanceGroup column.
- **title:** Sets the chart title.

The pie chart displays the proportion of flights for different distance groups (250-mile distance interval groups).

28.6 Create a Dash Application

```
app = dash.Dash(__name__)
```

28.6.1 Explanation

- `dash.Dash(__name__)` creates an instance of the Dash application.
- `__name__` helps Dash locate application resources correctly.

28.7 Define the Application Layout

The layout consists of:

1. A main title (`html.H1`)
2. A description (`html.P`)
3. A graph (`dcc.Graph`)

```
app.layout = html.Div(
    children=[
        html.H1(
            'Airline Dashboard',
            style={
                'textAlign': 'center',
                'color': '#503D36',
                'font-size': 40
            }
        ),
        html.P(
            'Proportion of distance group (250 mile distance interval group) by flights.',
            style={
                'textAlign': 'center',
                'color': '#F57241'
            }
        ),
        dcc.Graph(
            figure=fig
        )
    ]
)
```

```

    )
]
)
figure=fig passes the Plotly pie chart to the dashboard.

```

28.8 Complete Application Code

```

# Import required packages
import pandas as pd
import plotly.express as px
import dash
from dash import dcc
from dash import html

# Read the airline data into pandas dataframe
airline_data = pd.read_csv("8.1_airline_data.csv")

# Randomly sample 500 data points
data = airline_data.sample(n=500, random_state=42)

# Pie Chart Creation
fig = px.pie(
    data,
    values='Flights',
    names='DistanceGroup',
    title='Distance group proportion by flights'
)

# Create a Dash application
app = dash.Dash(__name__)

# Define application layout
app.layout = html.Div(
    children=[
        html.H1(
            'Airline Dashboard',
            style={
                'textAlign': 'center',
                'color': '#503D36',
                'font-size': 40
            }
        ),
        html.P(
            'Proportion of distance group (250 mile distance interval group) by flights.',
            style={
                'textAlign': 'center',
                'color': '#F57241'
            }
        ),
        dcc.Graph(
            figure=fig
        )
    ]
)

# Run the application
if __name__ == '__main__':
    app.run()

```

28.9 Running the Application

Run the following command in the terminal:

```
%run 9_Dash_Basics.py
```

Alternatively, run the Python file directly:

```
python 9_Dash_Basics.py
```

28.10 Key Takeaways

- **Dash** is used to build interactive web applications in Python.
- **HTML Components (html)** create the structure of the dashboard.
- **Core Components (dcc)** add interactive elements such as graphs and input widgets.
- **Plotly Express** is used to generate interactive charts.
- **dcc.Graph()** embeds Plotly figures into Dash applications.
- **html.Div()** acts as a container for organizing dashboard components.

29 Making Dashboards Interactive Using Callbacks

29.1 Understanding Callback Functions

A **callback function** is a Python function that is automatically called by Dash whenever an input component's property changes.

The callback function is decorated using the following decorator:

```
@app.callback
```

The purpose of this decorator is to tell Dash that whenever there is a change in the value of an input component:

1. The callback function should be executed automatically.
2. The output component in the application layout should be updated with the returned result.

29.2 Callback Function Workflow

The process of a callback can be summarized as follows:

1. User changes the value of an input component.
2. Dash automatically calls the callback function.
3. The callback function performs the required computation.
4. The result is returned.
5. The output component in the dashboard is updated.

29.3 Structure of a Callback Function

29.3.1 Step 1: Create the Function

Create a Python function that performs the operations required to produce the desired output.

29.3.2 Step 2: Decorate the Function

Decorate the function using:

```
@app.callback()
```

The decorator takes two main parameters:

- **Output:** Specifies the component and property that will receive the returned result.
- **Input:** Specifies the component and property whose change triggers the callback function.

In simple terms, the callback creates a connection between the input and output components.

29.4 Example: Top Ten Airline Carriers by Year

The objective is to extract the top ten airline carriers for a given year based on the number of flights.

When the user changes the year, the graph automatically updates.

29.5 Required Libraries

```
import pandas as pd
from dash import Dash, dcc, html
from dash.dependencies import Input, Output
```

29.5.1 New Import

```
from dash.dependencies import Input, Output
```

The Input and Output classes are used inside the callback function to establish connections between dashboard components.

29.6 Loading the Data

The airline data is loaded into a Pandas DataFrame.

```
df = pd.read_csv('airline_data.csv')
```

The DataFrame is loaded at the start of the application and can be accessed inside the callback function whenever needed.

29.7 Designing the Application Layout

29.7.1 Adding the Dashboard Title

The title is created using the HTML heading component:

```
html.H1()
```

The title's appearance can be customized using the `style` parameter.

29.7.2 Adding an Input Component

An HTML division and a text input component are added.

The input component has:

- Component ID: `input-yr`
- Property: `value`
- Default value: 2010

Example:

```
dcc.Input(  
    id='input-yr',  
    value='2010',  
    type='text'  
)
```

The `value` property of this component serves as the dashboard input.

29.7.3 Adding the Graph Component

A graph component is added inside an HTML division.

The graph component has:

- Component ID: `bar-plot`
- Property to update: `figure`

Example:

```
dcc.Graph(id='bar-plot')
```

29.8 Creating the Callback

The callback decorator connects the input component and the output component.

```
@app.callback(  
    Output(component_id='bar-plot',  
            component_property='figure'),  
    Input(component_id='input-yr',  
           component_property='value')
```

)

The keywords `component_id` and `component_property` are optional and are used mainly to improve readability.

29.9 Defining the Callback Function

```
def get_graph(year):  
    # Extract information using the entered year  
    # Generate the graph  
    return figure
```

The entered year becomes the input parameter of the function.

Using this year:

1. Required data is extracted from the DataFrame.
2. The graph is generated.
3. The application layout updates automatically.

29.10 Running the Application

Initially:

- Input Year = 2010

As the year changes, the graph updates automatically in real time.

29.11 Callback with Multiple Inputs

Dash callbacks can also accept multiple inputs.

The process is almost identical to a single-input callback, with only a few modifications.

29.11.1 Adding Another Input Component

A second text input is added with:

- Component ID: `input-ab`

Example:

```
dcc.Input(  
    id='input-ab',  
    value='AL',  
    type='text'  
)
```

29.11.2 Modifying the Callback Decorator

Multiple inputs are passed inside a list.

```
@app.callback(
    Output('bar-plot', 'figure'),
    [
        Input('input-yr', 'value'),
        Input('input-ab', 'value')
    ]
)
```

29.11.3 Updating the Callback Function

```
def get_graph(year, state):
    # Perform computations
    # Extract required information
    return figure
```

The function now accepts two parameters:

- Entered year
- Entered state

The dashboard updates whenever either input changes.

29.12 Example Output

Initial values:

- Year = 2010
- State = AL (Alabama)

As either the year or state changes, the graph updates automatically in parallel.

29.13 Key Takeaways

- A **callback function** is a Python function automatically called by Dash whenever an input component's property changes.
- The **@app.callback decorator** tells Dash when to execute the callback function.
- The callback function connects the brew list | grep python
- **Input** and **Output** components.
- The function performs computations and returns the desired result to the output component.
- Dash supports both **single-input** and **multiple-input** callbacks, enabling highly interactive dashboards.

30 Summary: Creating Dashboards with Plotly and Dash

30.1 Dash

- Dash is an open-source user interface Python library for creating reactive, web-based applications.
- It is easy to build graphical user interfaces (GUIs) using Dash because it abstracts (hides the complexity of) the underlying technologies required to create web applications.

30.2 Components of Dash

Dash consists of two main components:

- Core Components
- HTML Components

30.2.1 Dash Core Components

- The `dash_core_components` library provides higher-level interactive components.
- These components are generated using JavaScript, HTML, and CSS through the `React.js` library (a JavaScript library for building user interfaces).

30.2.2 Dash HTML Components

- The `dash_html_components` library contains a component for every HTML tag.
- It allows developers to create web page structures directly using Python code.

30.3 Callback Functions in Dash

- A callback function is a Python function that is automatically called by Dash whenever an input component's property changes.
- Callback functions enable applications to become reactive and interactive.

30.3.1 The `@app.callback` Decorator

- The `@app.callback` decorator is used to decorate (attach) a callback function.
- It tells Dash to automatically execute the callback function whenever there is a change in the value of an input component.

30.3.2 Working of Callback Functions

- The callback function takes input and output components as parameters.
- It performs the required operations and returns the desired result to the output component.

30.4 Key Takeaways

- Dash is an open-source Python framework for building interactive web applications.
- Dash simplifies GUI development by abstracting web technologies.
- Dash has two major component libraries:
 - `dash_core_components`
 - `dash_html_components`
- Callback functions make Dash applications interactive and reactive.
- The `@app.callback` decorator links input components to output components and automatically triggers updates whenever inputs change.

31 Common Issues in Dash Applications

31.1 Syntax Error: Invalid Syntax

31.1.1 Description

This error typically occurs when a Python function definition (such as `def update_input_container`) is flagged as invalid. The root cause is often an unclosed parenthesis `()` or square bracket `[]` in the layout code immediately preceding the function.

Dash uses a strict list-based hierarchy (`html.Div([...])`) to group child components, and a missing closing bracket breaks Python's syntax structure.

31.1.2 Example

INCORRECT: Missing a closing square bracket and parenthesis before the callback function

```
html.Div([
    html.Label("Select Statistics:"),
    dcc.Dropdown(
        id='dropdown-statistics',
        options=[{'label': 'Yearly Status', 'value': 'Yearly'}],
        value='Yearly'
    )
# <-- Missing ]) here causes a syntax error on the next line!

@app.callback(
    Output(component_id='output-container', component_property='children'),
    Input(component_id='dropdown-statistics', component_property='value')
)
def update_input_container(selected_statistics):
    # Function fails to parse due to the broken layout above
    pass
```

31.1.3 Solution

- Check the layout code immediately before the reported error.
- Ensure that all parentheses `()` and square brackets `[]` are properly closed.
- Verify the structure of nested components, such as `html.Div()` and `dcc.Dropdown()`.

31.2 Callback Error Updating Output

31.2.1 Description

This issue occurs when a callback fails to update the target component property (for example, `output-container.children`).

It is usually caused by:

- Improperly defined input or output variables.
- Incorrect ordering of Input objects.
- Mismatch between callback inputs and function parameters.
- Incorrect variable scope.

31.2.2 Example

CORRECT: Input and Output variables are correctly structured and scoped

```

@app.callback(
    Output(component_id='output-container', component_property='children'),
    [
        Input(component_id='dropdown-statistics', component_property='value'),
        Input(component_id='select-year', component_property='value')
    ]
)
def update_charts(selected_statistics, input_year):
    # Ensure both arguments match the Input definitions above in the correct order
    pass

```

31.2.3 Solution

- Ensure that every Input in `@app.callback` has a corresponding function parameter.
- Maintain the same order of function arguments as the Input definitions.
- Verify that component IDs and properties are correct.

31.3 Graph Not Displaying

31.3.1 Description

If the dashboard loads but the graphs are blank or missing, the issue usually lies in the conditional logic.

Common causes include:

- Incorrect conditional statements.
- Improper dataframe filtering.
- Mismatch between dropdown values and conditions.

31.3.2 Example

```

# CORRECT: Utilizing conditional statements to filter data properly before rendering
@app.callback(...)
def update_charts(selected_statistics, input_year):
    if selected_statistics == 'Recession Period':
        # logic for recession data
        pass

    elif (input_year and selected_statistics == 'Yearly Status'):
        # Correctly filtering the dataframe based on the selected year variable
        yearly_data = data[data['Year'] == input_year]

    # Proceed to build and return your graph figures

```

31.3.3 Solution

- Verify that dropdown values match the conditions exactly.
- Ensure the dataframe is filtered using the correct column names and variables.
- Check that the callback returns a valid graph object.

31.4 File Could Not Be Located Under the Project

31.4.1 Description

This error occurs when the script attempts to read a dataset or skeleton file that has not been downloaded into the active workspace directory.

31.4.2 Solution

Download the required file directly into your workspace.

```
wget https://Final_Project/DV0101EN-Final_Assign_Part_2_Questions.py
```

31.4.3 Recommendation

- Verify the current working directory.
- Ensure all required datasets and scripts are present before running the application.

31.5 Port 8050 Is in Use

31.5.1

31.5.2 Description

By default, Dash applications run on port 8050. If another application or a previous instance of the script is still running, Dash will generate a port conflict error.

31.5.3 Example

```
# CORRECT: Setting an alternate port (e.g., 8090) to bypass conflict on 8050
if __name__ == '__main__':
    app.run_server(port=8090)
```

31.5.4 Solution

- Stop the process currently using port 8050, or
- Run the application on a different available port.

31.5.5 Recommendation

Common alternative ports include:

- 8051
- 8060
- 8090
- 8888

Choose any unused port when starting the Dash application.